

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO
PAULO**

PEDRO HENRIQUE ALVES ROSENDO

SISTEMA SUPERVISÓRIO COM IoT PARA ANÁLISE DE UM MOTOR DC

**SÃO PAULO
2024**

PEDRO HENRIQUE ALVES ROSENDO

SISTEMA SUPERVISÓRIO COM IoT PARA ANÁLISE DE UM MOTOR DC

Trabalho de Conclusão de Curso apresentado ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, do Campus São Paulo, como requisito para obtenção do título de Bacharel em Engenharia de Controle e Automação.

Orientador: Prof. Dr. Caio Igor Gonçalves Chinelato.

São Paulo

2024

RESUMO

Nos ambientes industriais contemporâneos, a automação e a coleta de dados são fundamentais para maximizar a eficiência, a produtividade e a qualidade dos processos. Sistemas de aquisição de dados (DAQ – *Data Acquisition*) desempenham um papel central, permitindo monitorar em tempo real o desempenho de máquinas e equipamentos, o que possibilita a identificação de problemas potenciais antes que se tornem críticos. Com a análise contínua de variáveis como temperatura, corrente, tensão e vibração, esses sistemas permitem ajustes operacionais imediatos e a implementação de estratégias de manutenção preditiva, que reduzem significativamente o tempo de inatividade não programado e aumentam a vida útil dos equipamentos. Neste contexto, torna-se importante a comunicação em tempo real de equipamentos de chão de fábrica com sistemas externos via internet, nascendo assim, a internet das coisas (IoT – *Internet of Things*). A integração de dispositivos conectados e sensores inteligentes possibilita o monitoramento remoto e o controle preciso de máquinas e sistemas. Isso permite que as empresas ajustem parâmetros operacionais à distância, aumentem a automação dos processos e, ao mesmo tempo, garantam a coleta e análise de dados em tempo real. Além disso, a *IoT* permite o cruzamento de grandes volumes de dados operacionais com algoritmos de aprendizado de máquina, possibilitando uma otimização contínua e dinâmica dos processos, o que se traduz em maior eficiência energética, menores custos operacionais e melhor desempenho de sistemas fabris. Neste contexto, este trabalho apresenta um sistema capaz de fazer a leitura de sinais de um motor DC utilizando-se de uma plataforma desenvolvimento ESP32. Sendo assim, foram utilizados os sensores INA219 para leitura de tensão, corrente e potência, um *encoder* rotativo para leitura dos dados de rotação do motor, e um módulo ponte H para controlar a velocidade de rotação do motor DC. As variáveis são lidas a cada 200 ms e enviadas para a plataforma InfluxDB onde os dados são armazenados junto ao horário de sua leitura. Estes dados são, então, consultados pelo Grafana, que fica responsável por exibir os dados lidos em um sistema supervisório para o usuário.

Palavras-chave: Sistema de aquisição de dados; Sistema supervisório; internet das coisas; Grafana; InfluxDB; Monitoramento Remoto.

ABSTRACT

In contemporary industrial environments, automation and data collection are essential to maximize efficiency, productivity, and process quality. Data Acquisition (DAQ – *Data Acquisition*) systems play a central role by enabling real-time monitoring of machine and equipment performance, which allows for the identification of potential issues before they become critical. Through continuous analysis of variables such as temperature, current, voltage, and vibration, these systems enable immediate operational adjustments and the implementation of predictive maintenance strategies, significantly reducing unplanned downtime and increasing equipment lifespan. In this context, real-time communication between factory floor equipment and external systems via the internet becomes important, thus giving rise to the Internet of Things (IoT – *Internet of Things*). The integration of connected devices and intelligent sensors enables remote monitoring and precise control of machines and systems. This allows companies to adjust operational parameters remotely, increase process automation, and simultaneously ensure real-time data collection and analysis. Furthermore, IoT enables the cross-referencing of large volumes of operational data with machine learning algorithms, allowing for continuous and dynamic process optimization, resulting in higher energy efficiency, lower operational costs, and improved performance of manufacturing systems. Therefore, this work presents a system capable of reading signals of a DC motor using ESP32 development board. In this system, we applied INA219 sensors for reading voltage, current, and power, a rotary encoder to read rotation data, and an H-bridge module to control the DC motor's rotational speed. The variables are read every 200 ms and sent to the InfluxDB platform, where the data is stored along with the time of the reading. These data are then queried by Grafana, which is responsible for displaying the readings in a supervisory system for the user.

Keywords: Data Acquisition System; Supervisory System; Internet of Things; Grafana; InfluxDB; Remote Monitoring.

LISTA DE FIGURAS

Figura 1: Componentes de um motor DC.....	10
Figura 2: Blocos que compõe um sistema DAQ.....	13
Figura 3: Exemplo de um sistema supervisório.....	14
Figura 4: Dashboard de exemplo do Grafana.....	17
Figura 5: Diagrama de blocos funcional do sensor INA219.....	19
Figura 6: Diagrama de configuração para leitura de tensão do INA219.....	20
Figura 7: Modelo do motor DC utilizado para obtenção dos sinais.....	21
Figura 8: Estrutura interna de um encoder rotativo.....	21
Figura 9: Diferença de ângulos entre os sinais do encoder rotativo do motor DC.....	22
Figura 10: Diagrama de fluxo de dados adquiridos do motor DC.....	25
Figura 11: Pannel do board manager do Arduino IDE.....	26
Figura 12: Trecho do arquivo JSON de configuração da Arduino IDE para a plataforma ESP32.....	26
Figura 13: Tela de instalação do pacote de bibliotecas do ESP32.....	27
Figura 14: Exemplo de requisição HTTP enviada para escrita no InfluxDB.....	27
Figura 15: Página do github do client utilizado para comunicação com o InfluxDB.....	28
Figura 16: Código fonte no arquivo Adafruit_INA219.cpp.....	29
Figura 17: Frame extraído do vídeo utilizado para cálculo do tempo de uma única rotação.....	31
Figura 18: Topologia de testes para o sensor INA219.....	32
Figura 19: Topologia utilizada para os testes do módulo de ponte H L298N.....	33
Figura 20: Ondas do sinal gerado pelo encoder durante a rotação do motor.....	34
Figura 21: Tela de setup de usuário inicial no InfluxDB.....	35
Figura 22: Token de autenticação do InfluxDB.....	36
Figura 23: Caminho para adição de datasources.....	36
Figura 24: Configuração do InfluxDB como datasource no Grafana.....	37
Figura 25: Dashboard montado para apresentar os valores lidos do motor DC.....	40
Figura 26: Query utilizada para consulta do duty cycle.....	40
Figura 27: Query utilizada para consulta de tensão.....	40
Figura 28: Query utilizada para consulta de corrente elétrica.....	41
Figura 29: Query utilizada para exibição de rotações por minuto.....	41
Figura 30: Query utilizada para exibição de potência.....	42
Figura 31: Esquemático completo do sistema de aquisição de dados do motor DC.....	43
Figura 32: Figura 32: Visualização 3D da placa de circuito projetada para a leitura dos sinais do motor DC.....	44
Figura 33: Pannel de configuração de alerta do Grafana.....	45
Figura 34: Definição do alerta de corrente.....	46
Figura 35: Setup para testes com o módulo L298N.....	47
Figura 36: Leituras de tensão, corrente e potência medidas a partir do sensor INA219.....	48
Figura 37: Saída do script de contagem de pulsos.....	49

Figura 38: Dashboard do motor DC populado com as leituras do motor no Grafana.....	50
Figura 39: Alerta de corrente elevada disparado no Grafana.....	50

LISTA DE TABELAS

Tabela 1: Mapeamento entre endereços e posições dos jumpers no INA219.....	30
--	----

LISTA DE ABREVIATURAS E SIGLAS

DAQ	Data Acquisition
IHM	Interface Homem Máquina
DC	Direct Current
CSV	Comma Separated Values
ADC	Analog to Digital Converter
IoT	Internet Of Things
FEM	Força Eletromotriz
TSDB	Time Series Database
PWM	Pulse Width Modulation
JSON	JavaScript Object Notation
API	Application Programming Interface
RPM	Rotations per minute
V	Volts
IDE	Integrated Development Environment
YAML	YAML Ain't Markup Language

SUMÁRIO

1. Introdução	6
1.1. Justificativa	6
1.2. Objetivos	7
2. Revisão bibliográfica:	9
2.1. Motor DC	9
2.2. Controle de Velocidade e Torque	11
2.3. Tipos de perdas e eficiência no Motor DC	11
2.4. DAQ	12
2.5. Sistemas Supervisórios	13
2.6. IoT	14
2.7. Time Series Data, Time Series Databases e InfluxDB	16
2.8. Grafana	16
3. Desenvolvimento	19
3.1. Definição dos Sensores	19
3.2. Definição da placa processadora dos sinais dos sensores	22
3.3. Definição do driver de controle do motor	23
3.4. Definição das plataformas de envio e apresentação dos dados capturados	23
3.5. Configuração das bibliotecas utilizadas para programação do ESP32	25
3.6. Definição da biblioteca cliente do InfluxDB	27
3.7. Definição do cliente de comunicação com o INA219	29
3.8. Definição do número de pulsos por rotação do motor DC	30
3.9. Configuração do sensor INA219	32
3.10. Configuração da ponte H L298N	32
3.11. Configuração das plataformas InfluxDB e Grafana	34
3.12. Configuração do cliente do InfluxDB no ESP32	37
3.13. Definição da taxa de amostragem	38
3.14. Envio dos dados em lote para o InfluxDB	38
3.15. Configuração do dashboard no Grafana	39
3.16. Configuração de alerta no Grafana	45
4. Resultados	47
5. Conclusão	51
6. Referências:	53
Apêndice 1 – Sketch de testes do sensor INA219	56
Apêndice 2 – Sketch utilizado para geração do PWM em 80% para a gravação das rotações	57
Apêndice 3 – Sketch utilizado para determinar o número de pulsos por revolução do motor DC	58
Apêndice 4 – Sketch utilizado para testes do módulo de ponte H L298N	59
Apêndice 5 – Sketch de exemplo para conexão do cliente do InfluxDB no ESP32 ...	61
Apêndice 6 – Docker compose utilizado para configuração do Grafana e InfluxDB.	62
Apêndice 7 – Modelo JSON utilizado para importação das views no Grafana	63
Apêndice 8 – Definição do alerta exportado do Grafana no formato YAML	74
Apêndice 9 – Script para geração de dados para disparo do alerta em C#	76
Apêndice 10 – Código fonte final do sistema de aquisição de dados	77

1. Introdução

Até a década de 1980, a máquina de corrente contínua convencional (com escovas) era a escolha padrão para controle de velocidade ou torque, e um grande número delas ainda está em operação, apesar da queda na participação de mercado, que reflete a migração para motores de indução alimentados por inversores (HUGHES, 2006). Atualmente, essas máquinas elétricas continuam sendo objeto de estudo intensivo, especialmente em cursos de engenharia, sendo parte fundamental do aprendizado de estudantes ao redor do mundo. Como qualquer sistema dinâmico, o comportamento desses motores pode ser descrito por dois regimes distintos: o regime transitório e o regime permanente. (ALEXANDER e SADIKU, 2013).

O regime permanente é caracterizado por um estado de operação estável, no qual o sistema não sofre variações bruscas. Nesse regime, muito tempo após a aplicação de uma excitação externa, desconsiderando as perdas devido a fatores como atrito e aquecimento, o sistema manteria seu comportamento de forma indefinida. Por outro lado, o regime transitório descreve o comportamento do sistema logo após a aplicação de uma excitação, até que ele atinja o regime permanente. (ALEXANDER e SADIKU, 2013).

Embora seja possível descrever matematicamente o comportamento de muitos sistemas, a modelagem matemática nem sempre é simples, e muitas vezes é necessário validar o modelo por meio de medições práticas. Nesse contexto, sistemas de aquisição de dados (DAQ – *Data Acquisition*) desempenham um papel essencial.

Em indústrias de alto risco, como as químicas, de eletricidade e de cimento, pequenas negligências podem causar grandes perdas econômicas e sérios danos ambientais. A aquisição e o processamento de dados tornam-se fundamentais, e os requisitos de desempenho — como precisão do sistema e seu tamanho — variam conforme as diferentes áreas de aplicação. Os sistemas em questão possuem controle de aquisição de dados embutido, com interação online, o que torna estes sistemas mais confiáveis (PALIWAL, KHARDE, 2015). Neste cenário, os DAQ's permitem o monitoramento em tempo real de máquinas e equipamentos, possibilitando a detecção de problemas antes que se tornem críticos.

1.1. Justificativa

A análise contínua de variáveis como temperatura, corrente, tensão e vibração em motores permite ajustes operacionais imediatos e a implementação de estratégias de manutenção preditiva, contribuindo para a redução do tempo de inatividade não programado e aumentando a vida útil dos equipamentos e, com a evolução tecnológica, especialmente com a internet das coisas (IoT – *Internet*

of Things), transformou ainda mais esse panorama industrial. A integração de sensores inteligentes e dispositivos conectados proporciona uma nova dimensão de conectividade e controle remoto, permitindo que parâmetros operacionais sejam ajustados à distância, ao mesmo tempo em que facilita a coleta e análise de dados em tempo real. A IoT também possibilita a análise de grandes volumes de dados operacionais por meio de algoritmos de aprendizado de máquina, o que resulta em uma otimização contínua dos processos. Como resultado, as empresas podem alcançar uma maior eficiência energética, redução de custos operacionais e melhorias no desempenho geral dos sistemas. (PALIWAL, KHARDE, 2015).

A incorporação de tecnologias avançadas de automação e o uso da IoT representam uma mudança na forma como as indústrias operam. E este novo modelo oferece maior controle e competitividade assegurando o cumprimento de padrões de qualidade e segurança.

Além de sua aplicação em ambientes industriais, os DAQs apresentam uma ampla gama de utilizações nas áreas de controle e conversão de energia elétrica. No campo da conversão de energia, os DAQs desempenham um papel crucial ao permitir a análise do comportamento dinâmico das máquinas elétricas, tanto em regimes transitórios quanto permanentes. Já no estudo de sistemas de controle, esses sistemas são fundamentais no apoio ao desenvolvimento e validação de controladores, além de possibilitarem a implementação e avaliação de diversos algoritmos de controle. (PALIWAL, KHARDE, 2015).

1.2. Objetivos

O projeto apresentado neste trabalho visa desenvolver um sistema de aquisição de dados de baixo custo para motores de corrente contínua, com integração à nuvem através de IoT para a apresentação e exportação dos dados. O sistema lê os dados de tensão, corrente, potência e rotação em um motor DC a cada 200 ms e envia as leituras para um o InfluxDB, um *time series database*, no qual é possível exportar os dados em formato independente de plataforma, como arquivos CSV, que facilitam a análise em diversas ferramentas e sistemas de análise de dados.

Deste modo, o sistema fornece medições precisas tanto para o regime permanente quanto para o regime transitório do motor em estudo. As medições são apresentadas em *near real time* (o processamento e visualização dos dados possuem um pequeno atraso) em um sistema supervisório construído dentro do Grafana, permitindo o acompanhamento contínuo das variáveis de interesse e oferecendo uma visão detalhada do comportamento do motor durante sua operação.

Para que o objetivo geral deste projeto seja alcançado, são necessários o cumprimento dos seguintes objetivos específicos:

- Escolha do processador para leitura e integração dos dados com a internet;
- Escolha dos sensores para corrente, tensão e potência;
- Escolha do sensor para rotação do motor;
- Programação do processador para integrar com os sensores;
- Programação do processador para integrar com o InfluxDB;
- Configuração do Grafana para integração com o InfluxDB;
- Criação de um dashboard no Grafana para apresentação dos dados lidos dos sensores;
- Configuração de alerta exemplo para uma das variáveis lidas do motor no Grafana.

2. Revisão bibliográfica:

Neste capítulo serão apresentadas algumas considerações gerais sobre sistemas de aquisição de dados, máquinas elétricas, IoT e segurança cibernética a fim de contextualizar o leitor nos assuntos objeto de estudo deste trabalho.

2.1. Motor DC

Um motor de corrente contínua (DC – *Direct Current*) é uma máquina elétrica que pode ser descrita pelos seguintes componentes (HUGHES, 2006):

- Rotor (ou Armadura): O rotor é a parte móvel do motor e contém várias bobinas de fio enroladas em um núcleo ferromagnético. Quando a corrente passa por essas bobinas, um campo magnético é gerado que interage com o campo magnético estático do estator. A comutação, um processo crucial em motores DC com escovas, envolve a inversão da corrente nas bobinas do rotor conforme ele gira, garantindo que o torque seja aplicado de maneira contínua e consistente;

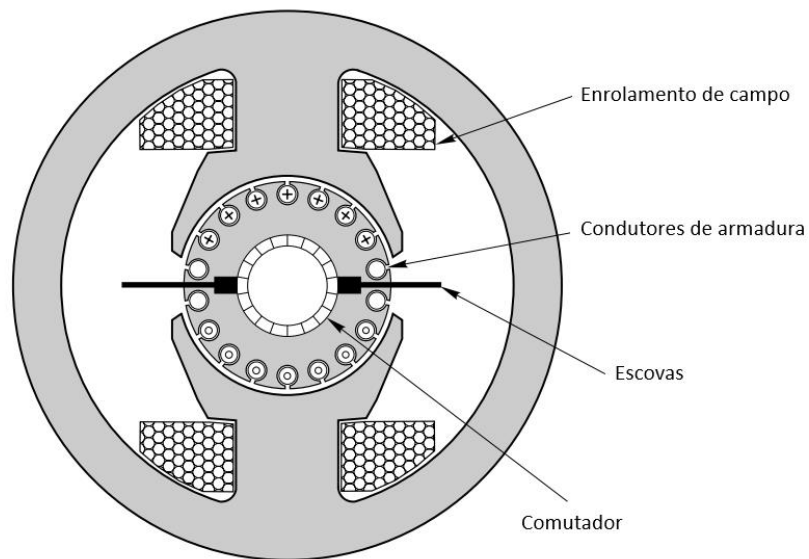
- Estator: O estator é a parte fixa do motor e fornece o campo magnético estático que interage com o rotor para gerar o torque rotacional. Nos motores DC, o estator pode ser composto por ímãs permanentes, que oferecem um campo magnético constante, ou por bobinas eletromagnéticas, que permitem ajustar o campo magnético. Em motores de campo bobinado, a capacidade de ajustar o campo magnético possibilita um controle mais preciso sobre o torque e a velocidade do motor;

- Comutador: No motor DC, o comutador é um componente essencial que atua como um interruptor rotativo. Ele reverte a direção da corrente no rotor, garantindo que o campo magnético do rotor se alinhe continuamente com o campo magnético do estator. Isso é crucial para a geração de torque contínuo e estável;

- Escovas: As escovas conduzem a corrente elétrica do circuito externo para o rotor através do comutador. Feitas de materiais como carvão ou grafite, elas oferecem boa condutividade e baixo atrito. No entanto, o contato constante entre as escovas e o comutador provoca desgaste, exigindo manutenção periódica para evitar falhas e garantir o funcionamento eficiente do motor (HUGHES, 2006).

Assim, a interação desses componentes e princípios proporciona o funcionamento eficiente e o desempenho confiável dos motores DC em diversas aplicações industriais. Motores DC são geralmente encontrados em aplicações que requerem menor potência. Nestas máquinas, o tradicional enrolamento utilizado na construção do motor é substituído pelo uso de um ímã permanente. Esta construção gera alguns benefícios como a maior facilidade de construção (pois não necessitam de uma excitação externa) e também dissipam menos potência para criar um campo magnético interno na máquina de corrente contínua. A Fig. 1 ilustra os componentes de um motor DC.

Figura 1: Componentes de um motor DC.



Fonte: (adaptado de: HUGHES, 2006).

O funcionamento do motor DC é fundamentado em dois princípios eletromagnéticos fundamentais: a Lei de Faraday da Indução Eletromagnética e a Lei de Lorentz. A Lei de Faraday descreve como uma corrente elétrica em um condutor gera um campo magnético, enquanto a Lei de Lorentz explica como uma corrente elétrica interage com um campo magnético para produzir uma força mecânica (HALLIDAY, RESNICK e WALKER, 2016)

De acordo com a Lei de Faraday, a variação no fluxo magnético através de uma bobina induz uma Força Eletromotriz (FEM). Em um motor DC, essa lei é aplicada à armadura (ou rotor), onde a corrente elétrica circula pelas bobinas, criando um campo magnético ao redor delas. Essa interação é crucial para o funcionamento do motor, pois é a base para a geração de movimento (FITZGERALD, KINGSLEY e UMANS, 2006).

A Lei de Lorentz, por sua vez, define a força que atua sobre um condutor (as bobinas do rotor) imerso em um campo magnético de acordo com a equação (1):

$$F = BIL, \quad (1)$$

onde F é a força de Lorentz, B é a densidade de fluxo magnético, I é a corrente elétrica que passa pelo fio condutor e L é o comprimento do condutor dentro do campo magnético. Essa força gerada é o que produz o torque necessário para girar o rotor, resultando no movimento rotacional do motor DC.

2.2. Controle de Velocidade e Torque

A velocidade de um motor DC pode ser ajustada variando-se a tensão de entrada ou alterando o campo magnético (no caso de motores com campo bobinado). O controle de torque pode ser obtido ajustando a corrente que passa pelas bobinas da armadura (HUGHES, 2006).

2.3. Tipos de perdas e eficiência no Motor DC

A eficiência de um motor DC é definida como a relação entre a potência mecânica de saída e a potência elétrica de entrada, como mostrado na equação (2):

$$\eta = \frac{P_o}{P_i} = \frac{T\omega}{VI}, \quad (2)$$

onde η é a eficiência do motor, P_o é a potência de saída do motor, P_i é a potência elétrica de entrada do motor, T é o torque gerado pelo motor, ω é a velocidade angular do motor, V é a tensão elétrica aplicada ao motor e I é a corrente elétrica consumida pelo motor.

As perdas no motor DC são inevitáveis e afetam sua eficiência global. Existem várias fontes de perda:

- Perdas Ôhmicas: Ocorrem devido à resistência nas bobinas da armadura e do campo;
- Perdas por Atrito: Ocorrem devido ao atrito entre as escovas e o comutador, além do atrito nos mancais;
- Perdas Magnéticas: Perdas no núcleo do motor devido à correntes parasitas e histerese no material ferromagnético.

A eficiência e a resposta de um motor DC são influenciadas por fatores como a constante de torque, resistência do enrolamento, indutância e tensões aplicadas. Além disso, a modelagem matemática do motor, que envolve equações diferenciais descrevendo a relação entre tensão, corrente, torque e velocidade angular, é crucial para prever o comportamento dinâmico do sistema em diferentes condições operacionais. Ao aplicar técnicas de controle moderno, como PWM e controladores PID otimizados, é possível ajustar com precisão a velocidade e o torque, mesmo sob variações de carga e flutuações de tensão.

Embora os motores DC apresentem desafios, como desgaste mecânico devido a comutadores e escovas, além de perdas por aquecimento e resistência, a incorporação de tecnologias modernas está transformando a maneira como esses motores operam. A automação industrial baseada em IoT aliada a sistemas de controle digital e realimentados em tempo real, tem elevado a confiabilidade e a eficiência desses motores a novos patamares. Sensores embarcados e algoritmos de controle adaptativo permitem ajustes dinâmicos nas variáveis de operação, garantindo maior precisão e prolongando a vida útil do equipamento.

2.4. DAQ

Os DAQs desempenham um papel crucial em ambientes industriais ao permitir a coleta, processamento e análise de dados de uma variedade de sensores e transdutores. Estes sistemas capturam sinais analógicos, como temperatura, pressão e vibração, e os convertem em dados digitais, que são então visualizados, analisados e armazenados para diversas finalidades. Através da coleta e análise desses dados, os sistemas DAQ auxiliam no monitoramento do desempenho dos equipamentos, na identificação de falhas potenciais e na otimização dos processos produtivos, tornando-se essenciais para a manutenção preditiva e para assegurar que os padrões de qualidade sejam atendidos. (TEKTRONIX, 2024).

Os sistemas DAQ consistem em diversos componentes interconectados que trabalham em conjunto para garantir uma coleta e processamento eficiente dos dados. Sensores são os primeiros elementos, responsáveis por medir variáveis físicas e convertê-las em sinais elétricos. Estes sinais, inicialmente analógicos, são então convertidos em dados digitais por conversores A/D (Analógico para Digital). (TEKTRONIX, 2024).

Finalmente, o software de análise e monitoramento processa e apresenta os dados em tempo real, gerando relatórios e enviando alertas automáticos em caso de condições anormais ou fora do padrão. (TEKTRONIX, 2024).

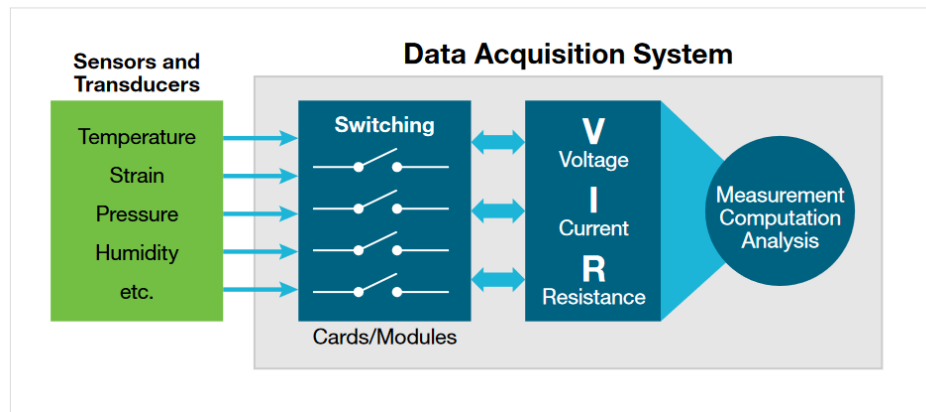
Alguns dos fatores relevantes na escolha de um DAQ são (TEKTRONIX, 2024):

- Resolução e acurácia: A acurácia é determinada pela resolução do conversor analógico digital (ADC) e a estabilidade de sua tensão de referência;
- Taxa de amostragem: Uma taxa de amostragem alta é essencial para aquisição de sinais de alta variação e para controle de sistemas dinâmicos;
- Condicionamento de sinais: É comum que sistemas de aquisição incluam filtros, amplificadores e a digitalização de diferentes tipos de sensores. É aplicado a fim de melhorar a qualidade dos sinais lidos, sendo essencial para adaptação de diversos tipos de sensores como termostatos, sensores de distorção e acelerômetros;
- Compatibilidade: Por se tratar de um sistema que une *software* e *hardware*, é essencial que o *software* fornecido tenha compatibilidade com sistemas operacionais comuns como *Windows* e *Linux*;
- Número de canais: Cada canal é responsável pela leitura de um sensor. Sistemas com mais canais são capazes de ler e interpretar mais sinais;
- Ambiente: Para ambientes hostis é importante utilizar um DAQ desenvolvido para tal proposito, pois isto prolonga a vida útil do equipamento;

- Custo: É necessário avaliar o custo de forma geral, desde a compra do equipamento até eventuais gastos com licenças de *softwares* proprietários.

A Fig. 2 ilustra os blocos funcionais que compõe um sistema DAQ.

Figura 2: Blocos que compõe um sistema DAQ.



Fonte: (TEKTRONIX, 2024).

Em ambientes industriais, os sistemas DAQ são aplicados em diversas áreas. Na manutenção preditiva, dados de vibração, temperatura e pressão coletados de equipamentos são analisados para identificar padrões que podem indicar falhas iminentes, permitindo a intervenção antes que ocorram falhas catastróficas. No controle de processos, o monitoramento contínuo de variáveis críticas permite ajustes automáticos que otimizam a produção e garantem a qualidade dos produtos (MOBLEY, 2002). Além disso, o monitoramento de equipamentos, possibilitado pela coleta contínua de dados, oferece informações valiosas para maximizar a eficiência operacional.

A integração dos motores DC com sistemas DAQ permite o monitoramento em tempo real, usando sensores de vibração, temperatura e velocidade para garantir o funcionamento eficiente e dentro dos parâmetros estabelecidos. Os dados desses sensores, quando integrados ao DAQ, podem ser utilizados para prever falhas, otimizar o desempenho e realizar ajustes automáticos, contribuindo para a melhoria contínua dos processos (MOBLEY, 2002).

2.5. Sistemas Supervisórios

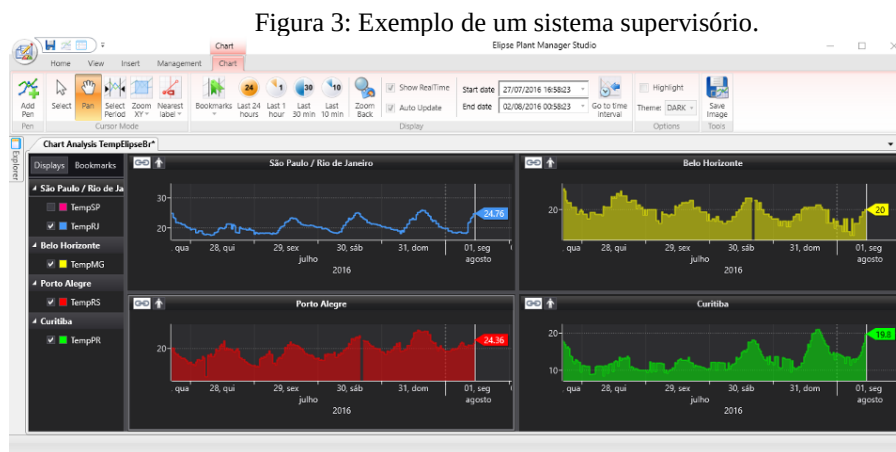
Nos processos industriais, geralmente, há a necessidade de centralizar as informações de forma a obter a maior quantidade de dados no menor tempo possível. Um sistema de supervisão é responsável pelo monitoramento das variáveis de controle do sistema, com o objetivo principal de fornecer suporte ao operador para controlar ou monitorar um processo automatizado de maneira mais ágil, permitindo a leitura das variáveis em tempo real e o gerenciamento do processo (ROGGIA e CARDOZO, 2016).

Com a evolução tecnológica, os computadores passaram a desempenhar um papel na gestão da aquisição e tratamento de dados, permitindo sua visualização em um monitor de vídeo e a geração de funções de controle complexas, abrangendo um mercado cada vez mais amplo (ROGGIA e CARDOZO, 2016)

As telas de supervisórios devem oferecer ao operador uma visão global do processo, incluindo os dados mais importantes para a operação do sistema monitorado. As principais características que um sistema de supervisão deve possuir são: (CARDOZO, 2016).

- Interface amigável com o operador;
- Geração de relatórios;
- Histórico de tendências das variáveis controladas em forma de gráficos ou tabelas;
- Facilidade para interação com outros aplicativos;
- Acesso automático a banco de dados;
- Acesso compartilhado e remoto;
- Conexão com a internet;
- Gerenciamento das condições de alarme.

A fig. 3 ilustra um o exemplo de sistema supervisório desenvolvido no *software Elipse*.



Fonte: (ELIPSE SOFTWARE, 2024).

2.6. IoT

Com a ascensão da Internet das Coisas (IoT), os DAQs passaram a desempenhar um papel crucial na transformação digital da indústria, marcando a evolução para a Indústria 4.0. A IoT revoluciona o cenário industrial ao conectar sensores e dispositivos a redes, permitindo a coleta, transmissão e análise de dados em tempo real. Esta conectividade aprimorada possibilita um controle mais eficiente e uma gestão mais inteligente dos processos industriais (STANKOVIC, 2014).

Uma das principais vantagens da integração da IoT com sistemas DAQ é o monitoramento remoto. Com essa tecnologia, operadores podem supervisionar o desempenho de máquinas e equipamentos a partir de qualquer localização, o que se torna particularmente valioso em operações que se estendem por múltiplos locais ou em ambientes de alto risco. Por exemplo, em fábricas com linhas de produção dispersas geograficamente ou em instalações de difícil acesso, a capacidade de monitorar e gerenciar operações remotamente aumenta a eficiência e a segurança.

Outra vantagem significativa é que a IoT permite que os sistemas ajustem automaticamente as variáveis do processo com base nos dados coletados. Isso significa que, ao detectar padrões ou condições fora do normal, o sistema pode realizar ajustes em tempo real para otimizar a produção. Por exemplo, se um sensor detectar uma variação na temperatura que possa comprometer a qualidade do produto, o sistema pode automaticamente modificar as condições do processo para corrigir a anomalia, sem a necessidade de intervenção humana. Essa capacidade de resposta rápida e autônoma não só melhora a eficiência operacional, mas também reduz o risco de erros humanos e aumenta a precisão do processo (HASSAN, 2020).

Além dessas vantagens, a integração de IoT com sistemas DAQ proporciona uma análise avançada de dados. Os dados coletados são armazenados e analisados usando técnicas de *big data* e *analytics*, permitindo uma visão detalhada sobre o desempenho dos equipamentos e a operação geral. Isso possibilita a detecção de tendências e padrões que podem não ser evidentes a partir de uma análise superficial. Tais dados ajudam a antecipar problemas, otimizar a manutenção e melhorar o planejamento estratégico. (HASSAN, 2020).

A IoT também contribui para a conectividade e a interoperabilidade dos sistemas industriais. Com a padronização dos protocolos de comunicação e a integração de diferentes tecnologias e sistemas, a IoT facilita a criação de uma rede de dados coesa e eficiente. Isso permite que diferentes componentes e sistemas dentro de uma instalação industrial comuniquem-se e operem em conjunto de maneira harmoniosa, promovendo uma integração mais fluida e um melhor fluxo de informações entre diferentes áreas da produção. (HASSAN, 2020).

Em resumo, a IoT não apenas transforma a maneira como os dados são adquiridos e analisados, mas também promove uma revolução na automação e na gestão industrial. A combinação de monitoramento remoto, automação inteligente, análise avançada de dados e conectividade integrada torna a Indústria 4.0 uma realidade, trazendo consigo melhorias significativas na eficiência, segurança e flexibilidade dos processos industriais.

2.7. *Time Series Data, Time Series Databases e InfluxDB*

Time series data são, no seu fundamento, uma sequência de pontos de dados coletados ou registrados em intervalos de tempo uniformes. Cada ponto de dado é essencialmente algo instantâneo, uma visão de uma condição ou estado específico em um momento particular no tempo. A principal diferença dos *time series data* está em sua temporalidade. Diferentemente dos dados transversais (dados coletados em um único momento de diversas fontes, onde cada ponto de dado pode ser uma entidade independente), os dados de séries temporais são, por natureza, sequenciais. Essa sequência é o que proporciona a oportunidade de realizar diversas formas de análise que não são possíveis com outros tipos de dados. Desde identificar sazonalidade nas vendas até detectar anomalias em dados de sensores (HERMANS, 2023).

À medida que a demanda por análises em tempo real cresceu, a necessidade de bancos de dados especializados para lidar com dados de séries temporais aumentou. Bancos de dados tradicionais frequentemente enfrentam dificuldades com o volume, variedade e velocidade dos dados de séries temporais. É nesse contexto que entram os *time series databases* (TSDB). Diferentemente dos bancos de dados convencionais, os TSDBs são otimizados para lidar com *time series data* de maneira eficiente, oferecendo funcionalidades como ingestão de dados em alta velocidade, capacidades de consulta em tempo real e métodos eficientes de compressão de dados (HERMANS, 2023).

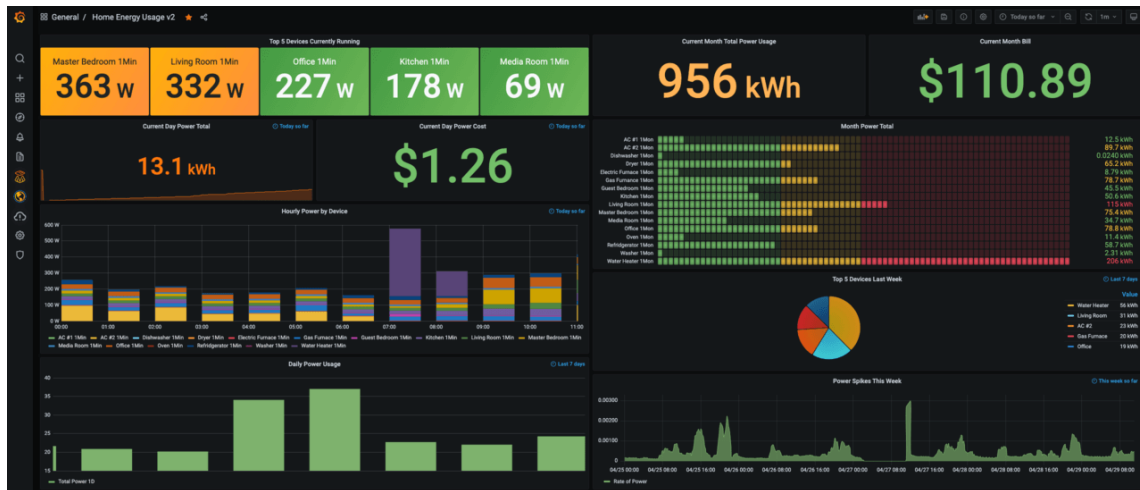
No contexto surgem, diversas soluções para os TSDBs, entre elas uma das mais populares é o *InfluxDB*. Este TSDB, além de *open-source*, foi projetado especificamente para lidar com grandes volumes de dados temporais, oferecendo uma arquitetura escalável, o que permite ajustar os recursos computacionais de acordo com a demanda além de adotar uma abordagem distribuída, possibilitando a criação de múltiplos nós computacionais. Essa arquitetura garante a alta disponibilidade e resiliência dos dados, de forma que, mesmo em caso de falha de uma das máquinas que hospeda o banco, os dados permanecem intactos e acessíveis, além de possuir uma linguagem de domínio específica própria (InfluxQL) para suas consultas e análise dos dados (HERMANS, 2023).

2.8. Grafana

O Grafana é uma plataforma que permite consultar, visualizar, gerar alertas e explorar métricas (dados gerados com base nas atividades de um sistema), *logs* (mensagens de eventos gerados com base nas atividades em um sistema) e *traces* (identificadores de correlação para rastreamento de ações dentro de um sistema), independentemente de onde estejam armazenados. Através de *plugins* de fontes de dados, o Grafana oferece integração com uma ampla variedade de ferramentas, incluindo

os TSDBs. No Grafana, os dados são exibidos em *dashboards* dinâmicos e interativos, utilizando gráficos e visualizações que permitem a monitorização e análise de praticamente qualquer tipo de sistema que produza métricas. (SALITURO, 2023). A Fig. 4 ilustra um *dashboard* do Grafana de exemplo.

Figura 4: *Dashboard* de exemplo do Grafana.



Fonte: (GRAFANA LABS, 2024).

Embora existam várias ferramentas de análise de dados poderosas no mercado que desempenham essas funções, o Grafana possui uma série de recursos que o tornam uma escolha interessante (SALITURO, 2023).

- Rápido: O *backend* (código responsável por acessar as fontes de dados externas) do Grafana é escrito na linguagem Go, desenvolvida pelo Google, o que o torna extremamente eficiente na consulta a fontes de dados ou ao alimentar milhares de pontos de dados em múltiplos *dashboards*;
- Aberto: O *Grafana* suporta um modelo de *plugins* para seus painéis de *dashboard* e fontes de dados que cresce constantemente à medida que a comunidade do Grafana contribui para o projeto;
- Extensível: O *Grafana* utiliza uma biblioteca de visualização de dados chamada D3, o que permite que tenhamos maior controle sobre a maioria dos elementos gráficos, incluindo eixos, linhas, pontos, preenchimentos, anotações e legendas;
- Versátil: O *Grafana* não está vinculado a uma tecnologia específica de banco de dados, podendo suportar uma variedade de fontes de dados que vão desde bancos de dados relacionais tradicionais, até TSDBs como o *InfluxDB*. Além disso, cada gráfico pode exibir dados de várias fontes de dados;
- *Open-source*: O *Grafana* OSS está disponível sob a licença de código aberto da Apache.

Além de ser uma ferramenta de visualização dos dados, o Grafana possui a funcionalidade de alertas. Os alertas no Grafana se baseiam principalmente em quatro componentes: *alert rules*, *labels*, *notification policies* e *contact points* (SALITURO, 2023):

- As *alert rules* são responsáveis pelo acionamento dos alertas do Grafana, pois, embora seja possível enviar diversas métricas para o Grafana, sem a definição de um *alert rule*, não haverá alerta quando algum parâmetro estiver fora do esperado;

- Os *labels* são metadados que o Grafana usa para identificar diferentes *alert rules*;

- As *notification policies* são o controle de tráfego para os alertas. Elas utilizam *templates* para definir o conteúdo das mensagens enviadas para os *contact points*. Atuam em conjunto com as *labels* que são associadas para montar quais mensagens enviadas para os *contact points*;

- Os *contact points* integram o sistema de alertas do Grafana com diversos destinos de notificação, como *email* ou *chat*. Um *contact point* é essencialmente um *plugin* do Grafana configurado para se comunicar com um serviço de destino de notificação.

3. Desenvolvimento

A primeira etapa na construção do DAQ consistiu na determinação das grandezas a serem monitoradas. Para este projeto, as variáveis de interesse foram:

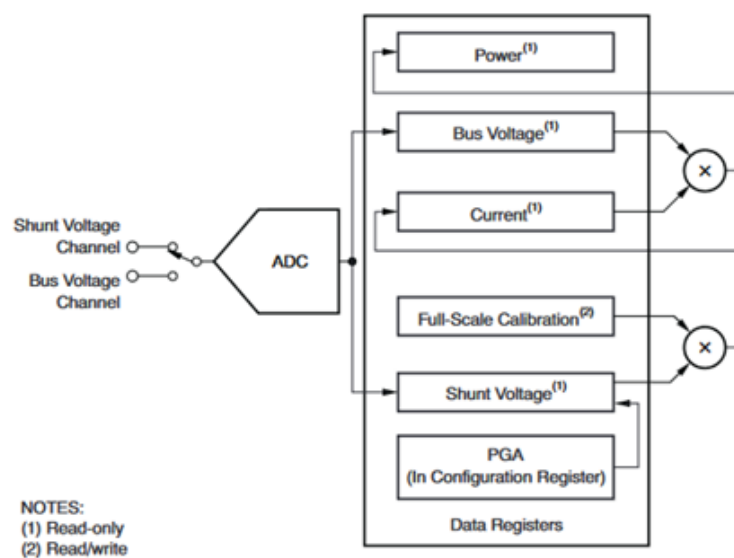
- Tensão do motor;
- Corrente do motor;
- Potência do motor;
- Quantidade de rotações do motor.

A placa de desenvolvimento, por sua vez, é configurada com base nos sinais a serem medidos, na taxa de amostragem e em outros requisitos funcionais, tais como o método de transmissão dos dados para o software responsável pela apresentação das medições dos sensores.

3.1. Definição dos Sensores

Para a medição de tensão, corrente e potência aplicadas ao motor, foi escolhido o sensor INA219, que é um amplificador digital de detecção de corrente com interface compatível com *I2C* e *SMBus*. Este sensor possui internamente um conversor analógico-digital de 12 bits que oferece leituras digitais de tensão (de até 26V), corrente (de até 3,2A) e potência, essenciais para a tomada de decisões precisas em sistemas de controle (TEXAS INSTRUMENTS). O diagrama de blocos funcional do sensor está ilustrado na Fig. 5.

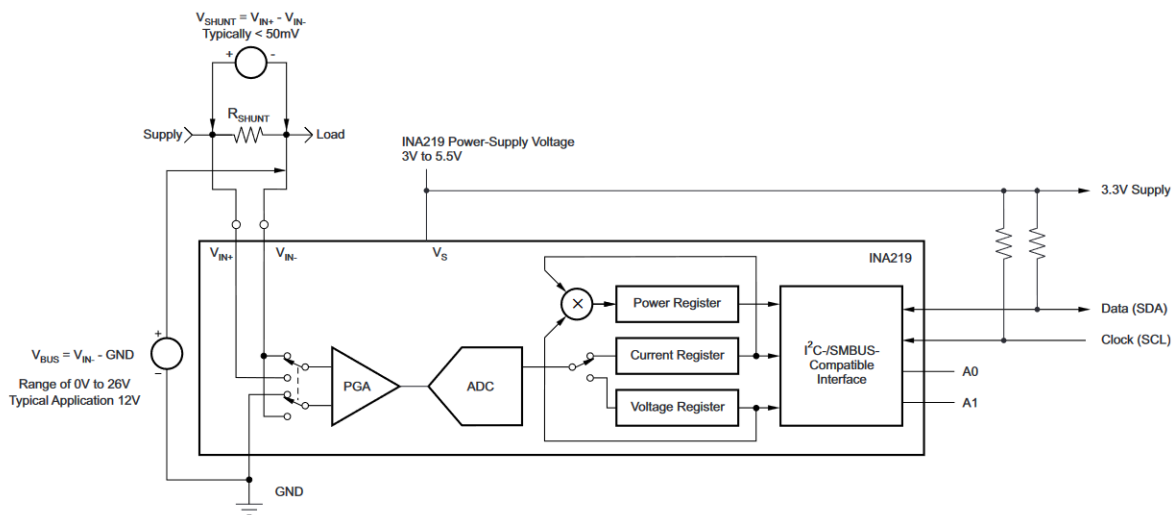
Figura 5: Diagrama de blocos funcional do sensor INA219.



Fonte: (TEXAS INSTRUMENTS, 2024).

O INA219 opera com base em um resistor de *shunt* localizado no barramento de interesse. O sensor realiza medições de tensão e corrente com base na tensão de queda através do *shunt*. O sensor possui registradores responsáveis pela configuração dos modos de aquisição dos sinais, permitindo as chamadas conversões contínuas ou conversões baseadas em eventos. Além disso, o sensor permite a paralelização dos processos de leitura e cálculo de tensão e corrente nos registradores, reduzindo o tempo entre as medições e possibilitando uma maior taxa de aquisição (TEXAS INSTRUMENTS, 2024). A Fig. 6 apresenta o diagrama do INA219 configurado para leitura do barramento de tensão a partir do resistor *shunt*.

Figura 6: Diagrama de configuração para leitura de tensão do INA219.



Fonte: (TEXAS INSTRUMENTS, 2024).

Por possuir uma função de Amplificador de Ganho Programável que permite expandir a faixa de medição da tensão do *shunt*, este sensor possibilita a calibração do sensor para medições de sinais com diferentes amplitudes, adaptando-se às necessidades de diferentes sistemas (TEXAS INSTRUMENTS, 2024).

Para este projeto, optou-se por utilizar um motor DC de 6V como mostrado na Fig. 7, modelo amplamente empregado em sistemas robóticos. Este modelo de motor se mostrou adequado para nossos propósitos, pois já incorpora um *encoder* rotativo, que é utilizado para monitorar tanto a velocidade de rotação do eixo quanto a posição do eixo em relação a uma referência.

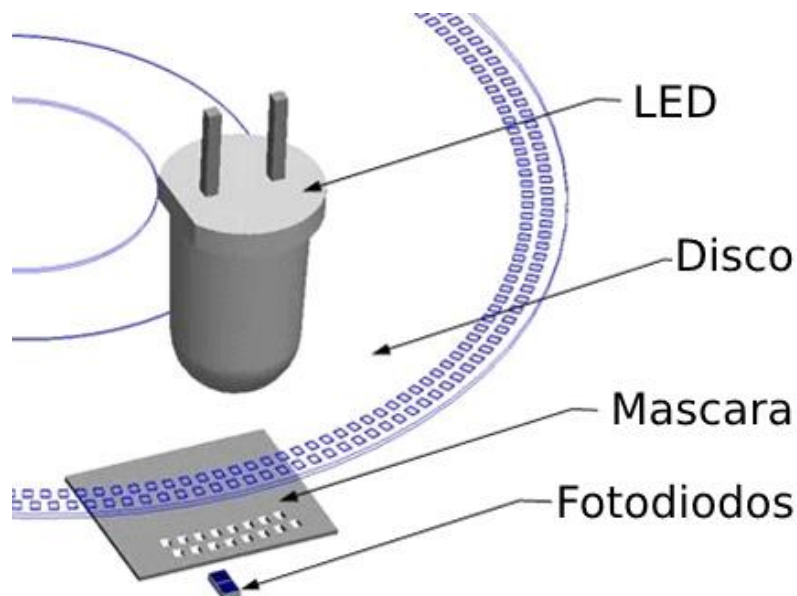
Figura 7: Modelo do motor DC utilizado para obtenção dos sinais.



Fonte: (ARDUINO, 2024).

No *encoder* presente nesta topologia, a medição das rotações é realizada com base em quatro componentes: uma fonte de luz (LED), um disco fixado ao eixo de rotação do motor, uma máscara e um par de sensores fotossensíveis. À medida que o eixo do motor se desloca, o espaçamento entre a máscara (janelas) faz com que pulsos de luz atinjam o sensor fotossensível em pulsos. Cada pulso aplicado ao sensor fotossensível gera uma mudança no sinal elétrico lido neste, indicando o movimento do motor. Com base na quantidade de pulsos gerados, é possível realizar a medição do ângulo de movimento da haste do motor e, conseqüentemente, o deslocamento angular deste (DYNAPAR, 2024). O princípio básico de funcionamento do encoder integrado ao motor escolhido pode ser observado na Fig. 8.

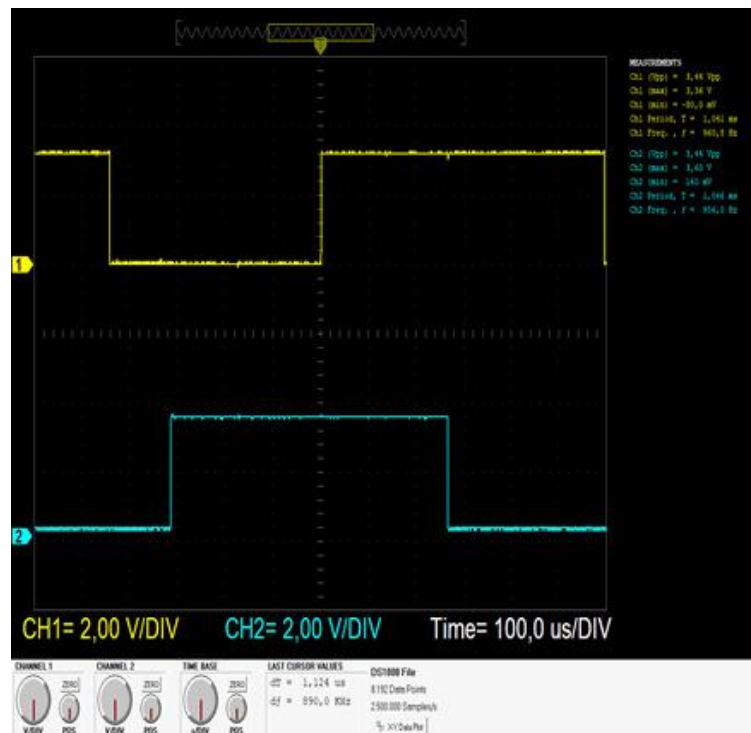
Figura 8: Estrutura interna de um *encoder* rotativo.



(Fonte: Adaptado de DYNAPAR, 2024).

Devido à diferença entre as janelas na construção do disco, o sensor fotossensível apresentará uma diferença entre os sinais lidos por cada um dos sensores. Essa diferença é denominada quadratura, pois um dos sinais estará sempre defasado em 90° em relação ao outro. O sinal obtido em um dos testes de funcionamento do motor pode ser observado na Fig. 9.

Figura 9: Diferença de ângulos entre os sinais do *encoder* rotativo do motor DC.



Fonte: Autor.

3.2. Definição da placa processadora dos sinais dos sensores

Uma vez definidos os sensores a serem utilizados na captura dos sinais do motor DC, era necessário dispor de uma plataforma de custo acessível com as seguintes características:

- Suporte à comunicação utilizando o protocolo I2C (necessário para comunicação com o sensor INA219);
- Suporte a interrupções externas (necessário para processar adequadamente os sinais gerados pelos pulsos no encoder rotativo);
- Suporte a bibliotecas utilizadas para plataformas de *cloud* (necessário para evitar perda de dados no reset da placa);

- Suporte à comunicação com a internet (utilizado para o envio dos sinais lidos para a plataforma responsável por apresentar os dados aos usuários).

Com base nos requisitos apresentados, optou-se pela utilização da plataforma ESP32, mais especificamente pelo modelo ESP32-WROOM-32, embarcado em uma placa do modelo ESP32 Dev Kit V1. Este modelo, além de atender aos requisitos funcionais, é suportado pela plataforma Arduino, possibilitando sua utilização em conjunto com o robusto conjunto de funções e bibliotecas oferecidas por essa plataforma. O ESP32 é considerado uma placa de alto desempenho, adequada para aplicações com acesso remoto e IoT. Ela é equipada com o processador Xtensa Dual-Core LX6 de 32 bits, com *clock* máximo de 240 MHz, 520 KB de memória RAM, 4 MB de memória flash, comunicação *Wifi* e *Bluetooth*, e 11 pinos GPIO, entre outros recursos (ESP32, 2024).

3.3. Definição do driver de controle do motor

Como o microcontrolador ESP32 não foi projetado para controlar cargas de alta potência, é necessário utilizar um driver de potência para realizar essa função. Para isso, optou-se pelo uso do *driver* de ponte H L298N, amplamente utilizado no controle de motores DC e motores de passo em sistemas embarcados. Uma das principais vantagens desse *driver* é sua capacidade de controlar tanto a direção quanto a velocidade dos motores, utilizando sinais de controle simples, como modulação por largura de pulso (*PWM – Pulse Width Modulation*) para ajuste de velocidade. Além disso, o L298N permite o controle do motor em ambos os sentidos de rotação, o que amplia sua versatilidade. O *driver* também é capaz de suportar correntes de até 2A em cada um de seus dois canais e operar em uma faixa de tensões que vai de 3,3V a 46V. Adicionalmente, o L298N conta com proteções contra sobrecarga.

3.4. Definição das plataformas de envio e apresentação dos dados capturados

Para a captura e apresentação dos dados foram testadas uma gama de plataformas de *cloud* capazes de salvar e apresentar os dados aos usuários do sistema. Algumas das plataformas testadas foram:

- AWS Cloudwatch metrics: serviço de métricas da *cloud* da *Amazon*. Possibilita uma integração e exportação dos dados de forma simples com agregação dos dados em intervalo de até 1

s. Foi descartada porque a visualização dos dados depende do gerenciamento de permissões (*roles*) dentro das contas na AWS, fugindo do escopo deste trabalho.

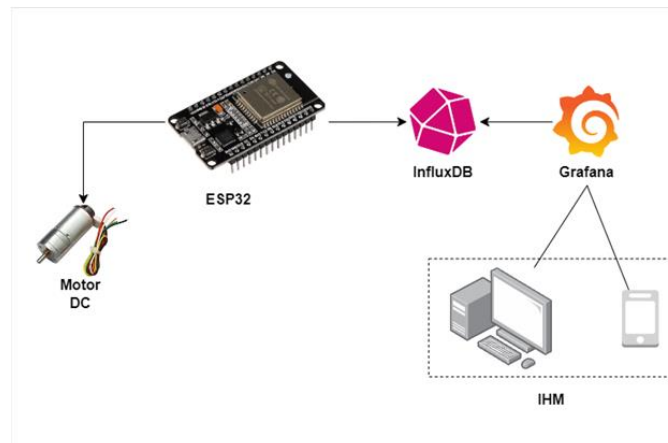
- *Blynk IoT Cloud*: plataforma de desenvolvimento e gerenciamento de aplicações IoT. Oferece infraestrutura na nuvem para controlar e monitorar remotamente dispositivos através de uma interface gráfica gerada por *Low Code* (abordagem de desenvolvimento de *software* que usa interfaces visuais e pouco código para criar aplicações rapidamente). Suporta diferentes protocolos e *hardwares* de prototipagem. Esta plataforma foi descartada por não dar suporte a extração dos dados enviados para a *cloud* no plano gratuito.

- *Arduino IoT Cloud*: Permite a conexão, monitoramento e controle de dispositivos Arduino através de uma interface web de fácil integração. Foi descartada por limitar a quantidade de variáveis que podem ser enviadas para a plataforma por projeto.

- *InfluxDB e Grafana*: Nesta arquitetura, o InfluxDB desempenha o papel de um TSDB, recebendo dados enviados por meio de requisições no protocolo HTTP (HERMANS, 2023), que contêm as medições realizadas pelos sensores. Cada conjunto de dados registrado no InfluxDB é representado por uma *tupla*, composta pelo *timestamp* (indicando o momento da leitura), o valor obtido e metadados - geralmente contendo informações como, por exemplo, o *MAC address* da origem da leitura. Por sua vez, o Grafana é a ferramenta responsável pela visualização desses dados, exibindo-os em um painel de fácil interpretação. A comunicação entre o InfluxDB e o Grafana ocorre através do protocolo HTTP, utilizando o método de consulta conhecido como *polling*, no qual o Grafana realiza requisições periódicas ao InfluxDB para obter as informações mais recentes.

Embora todas as plataformas mencionadas tenham sido avaliadas durante o desenvolvimento do projeto, optou-se pelo uso conjunto do InfluxDB e do Grafana para a implementação final. Essa escolha se deve, principalmente, à integração gratuita entre as duas ferramentas, bem como ao fato de que o InfluxDB permite a extração dos dados de séries temporais sem a necessidade de um plano pago, o que proporciona uma solução mais acessível e eficiente para as necessidades do projeto. A Fig. 10 ilustra o fluxo dos dados nesta topologia.

Figura 10: Diagrama de fluxo de dados adquiridos do motor DC.



Fonte: Autor.

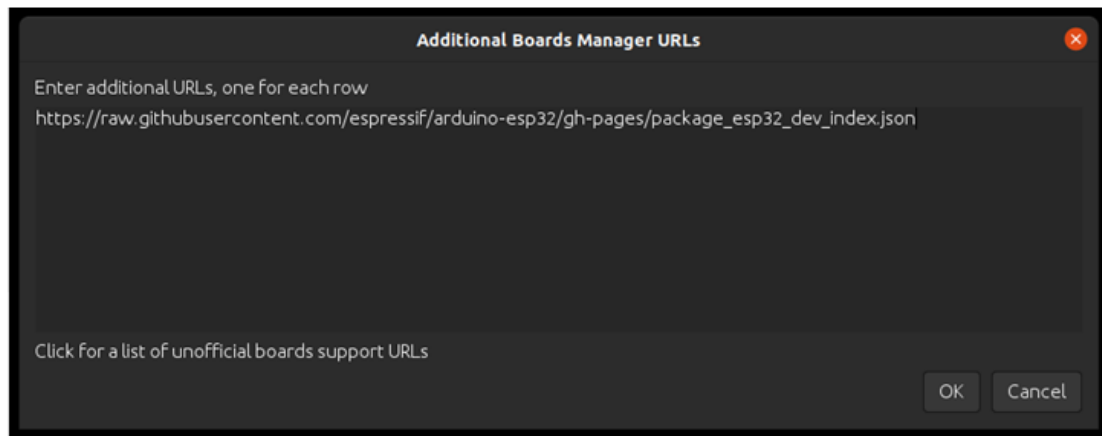
Ao analisar o fluxo de dados, é possível perceber que as informações do motor são inicialmente capturadas e lidas pela placa ESP32, que se encarrega de enviar esses dados para o InfluxDB. Posteriormente, o Grafana consulta o InfluxDB para a construção do painel de visualização. Esse painel pode ser acessado remotamente, por meio da internet, tanto em computadores quanto em dispositivos móveis, como *smartphones*.

3.5. Configuração das bibliotecas utilizadas para programação do ESP32

O ESP32 DEV Kit V1 é uma plataforma embarcada amplamente adotada em diversos projetos ao redor do mundo. Devido à sua popularidade, a empresa multinacional *Espressif* desenvolveu um conjunto de bibliotecas que facilitam a programação da placa ESP32 utilizando o Arduino IDE. Para estabelecer a compatibilidade entre a IDE e a placa, foi necessário seguir uma série de procedimentos descrito na documentação da *Espressif* para adicionar a referência a um arquivo de configuração contendo a definição de todas as placas suportadas pelo conjunto de bibliotecas e uma série de metadados referenciando arquivos utilizados pela Arduino IDE ao fazer a compilação (como, por exemplo, arquivos de cabeçalho). Os passos para a configuração são descritos a seguir:

- Adicionar o link para o pacote do ESP32 no Arduino IDE: Por se tratar de um pacote de bibliotecas não referenciadas por padrão pelo Arduino IDE, é necessário acessar o menu de preferências da Arduino IDE e referenciar o link do pacote na lista de *Additional Board Manager*, que permite instalar e gerenciar suporte à diferentes placas de *hardware* para programação, permitindo o suporte às placas ESP32. A Fig. 11 exemplifica a tela de adição de referência para as placas ESP32 dentro do Arduino IDE.

Figura 11: Pannel do *board manager* do Arduino IDE.



Fonte: (ESPRESSIF SYSTEMS, 2024).

Este link direciona para um arquivo no formato *JSON* (formato de troca de dados baseado em texto que usa a estrutura de chave-valor para representar objetos e arrays) que contém a declaração das plataformas suportadas pelo pacote de bibliotecas, além de uma referência aos *drivers* utilizados pela IDE para a compilação do projeto. A Fig. 12 apresenta um trecho deste arquivo, com as respectivas declarações, a título ilustrativo.

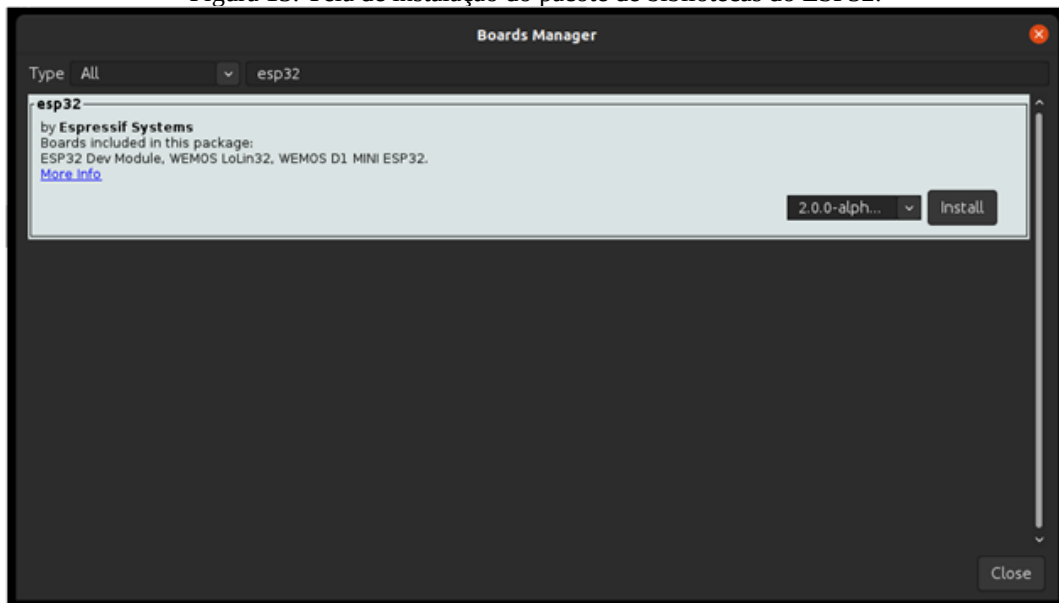
Figura 12: Trecho do arquivo JSON de configuração da Arduino IDE para a plataforma ESP32.

```
"tools": {
  {
    "name": "esp32-arduino-libs",
    "version": "idf-release_v5.1-632e0c2a",
    "systems": [
      {
        "host": "i686-mingw32",
        "url": "https://github.com/espressif/esp32-arduino-lib-builder/releases/download/idf-release_v5.1/esp32-arduino-libs-idf-release_v5.1-632e0c2a.zip",
        "archiveFileName": "esp32-arduino-libs-idf-release_v5.1-632e0c2a.zip",
        "checksum": "SHA-256:41f67e1c11f68b57d651955c93b63d6a8d35808ce6aff6ba3d1e1476178758f2",
        "size": "309895581"
      },
      {
        "host": "x86_64-mingw32",
        "url": "https://github.com/espressif/esp32-arduino-lib-builder/releases/download/idf-release_v5.1/esp32-arduino-libs-idf-release_v5.1-632e0c2a.zip",
        "archiveFileName": "esp32-arduino-libs-idf-release_v5.1-632e0c2a.zip",
        "checksum": "SHA-256:41f67e1c11f68b57d651955c93b63d6a8d35808ce6aff6ba3d1e1476178758f2",
        "size": "309895581"
      },
      {
        "host": "arm64-apple-darwin",
        "url": "https://github.com/espressif/esp32-arduino-lib-builder/releases/download/idf-release_v5.1/esp32-arduino-libs-idf-release_v5.1-632e0c2a.zip",
        "archiveFileName": "esp32-arduino-libs-idf-release_v5.1-632e0c2a.zip",
        "checksum": "SHA-256:41f67e1c11f68b57d651955c93b63d6a8d35808ce6aff6ba3d1e1476178758f2",
        "size": "309895581"
      }
    ]
  }
},
```

Fonte: (ESPRESSIF SYSTEMS, 2024).

- Instalar os pacotes do ESP32 no Arduino IDE: Após a adição da referência para o projeto de bibliotecas, é necessário instalar os *drivers* da placa. Como as referências para as bibliotecas já foram configuradas na etapa anterior, basta buscar por “esp32” na lista do *Board Manager* e concluir o processo de instalação. A Fig. 13 ilustra este procedimento.

Figura 13: Tela de instalação do pacote de bibliotecas do ESP32.



Fonte: (ESPRESSIF SYSTEMS, 2024).

3.6. Definição da biblioteca cliente do InfluxDB

A comunicação entre a placa de aquisição de sinais e o InfluxDB é realizada por meio do protocolo HTTP, com as operações descritas em uma API (conjunto de ações que permite a comunicação entre sistemas ou aplicativos, facilitando o acesso a funcionalidades ou dados) exposta pelo InfluxDB (INFLUXDATA, 2024). As operações implementadas pela API incluem consulta, escrita e exclusão tanto dos dados das séries temporais quanto dos *buckets* (unidades de segregação de séries temporais utilizadas pelo InfluxDB). A Fig. 14 apresenta as informações enviadas para uma requisição de escrita do InfluxDB a nível do protocolo HTTP.

Figura 14: Exemplo de requisição HTTP enviada para escrita no InfluxDB.

```
POST /api/v2/write?org=<ORG_ID>&bucket=<BUCKET_NAME>&precision=ms HTTP/1.1
Host: localhost:8086
Authorization: Token <INFLUXDB API TOKEN>
Content-Type: text/plain; charset=utf-8
Accept: application/json
Content-Length: <TAMANHO_DO_CORPO_DA_REQUISIÇÃO>

airSensors,sensor_id=TLM0201 temperature=73.970,humidity=35.231,co=0.484 1732036500000
airSensors,sensor_id=TLM0202 temperature=75.300,humidity=35.651,co=0.514 1732036501000
```

Fonte: (Adaptado de INFLUXDATA, 2024).

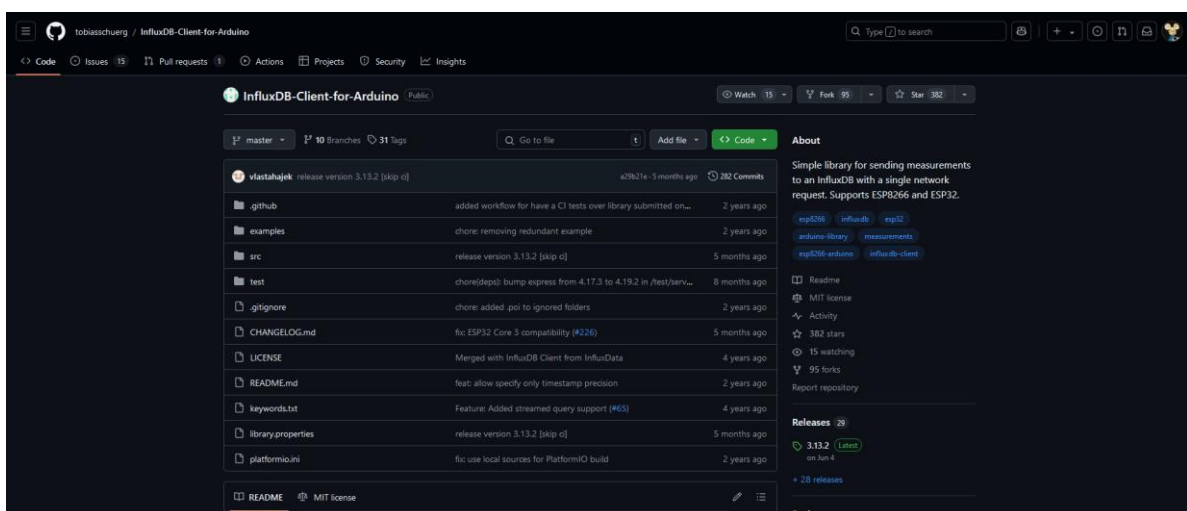
Na Fig. 14, estão destacados alguns elementos essenciais para a comunicação com o InfluxDB. O primeiro aspecto a ser observado é o método HTTP utilizado para a operação. Como se trata de

uma operação de escrita, em que o corpo da requisição contém os dados a serem inseridos, a API adota o método POST para esse tipo de transação. O primeiro parâmetro de consulta (*query parameter*), denominado "org", deve conter o identificador único da organização. Esse valor é fornecido pelo InfluxDB ao acessar o painel de configuração do sistema. Em seguida, o parâmetro "bucket", que representa a unidade lógica de organização dos dados, sendo utilizado para separar os valores lidos. Por fim, o último parâmetro de consulta, "precision", é utilizado para informar ao InfluxDB que os dados enviados na requisição possuem um *timestamp* no formato *epoch*, referenciado em milissegundos. Ainda, a requisição precisa conter um cabeçalho de "Authorization" incluindo o *token* de autorização criado pelo InfluxDB e atrelado a uma organização.

Além dos parâmetros que compõem os cabeçalhos e a *uri* da requisição, os dados são transmitidos no corpo da mesma utilizando um formato específico do InfluxDB, denominado *line protocol*. Esse protocolo é uma maneira eficiente e compacta de representar *time series data*, e, além de ser facilmente interpretado por seres humanos, é facilmente processado por *software*, facilitando a inserção de grandes volumes de informações (HERMANS, 2023). Outra vantagem do envio dos dados no formato *line protocol* é que podemos enviar diversas leituras por vez, o que diminui a sobrecarga associada à abertura de diversas conexões HTTP para cada transação, o que pode impactar a eficiência do processo (HERMANS, 2023).

Embora a API seja relativamente simples, a configuração de um cliente HTTP na placa não é uma tarefa trivial. Por isso, optou-se por utilizar a biblioteca de comunicação recomendada pela própria *InfluxData* (empresa responsável pelo desenvolvimento do InfluxDB) para facilitar as interações com a plataforma Arduino. A Fig. 15 apresenta a página do github do cliente utilizado para comunicação com o InfluxDB.

Figura 15: Página do *github* do *client* utilizado para comunicação com o InfluxDB.



Fonte: (SCHUERGER, 2024).

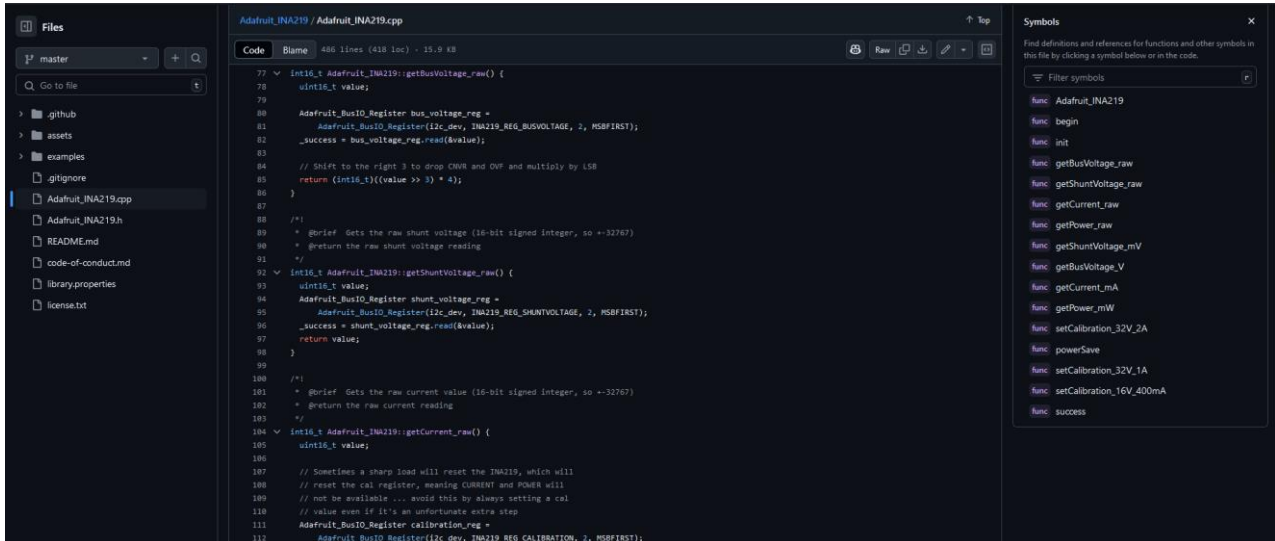
Dessa forma, foram utilizadas as classes do cliente para abstrair as operações descritas pela API.

3.7. Definição do cliente de comunicação com o INA219

Para a comunicação com o sensor INA219, utilizado para medir tensão, corrente e potência no motor, foi empregado o cliente de comunicação disponibilizado pela *Adafruit* (empresa fabricante de diversos módulos para sistemas embarcados, dentre ele o INA219). Assim como o cliente do InfluxDB, este cliente abstrai a comunicação I2C com o sensor, simplificando a implementação da leitura.

O cliente de comunicação da Adafruit é composto majoritariamente por uma classe C++ chamada *Adafruit_INA219*. O objeto criado a partir desta classe expõe diversos métodos para calibragem do PGA interno do INA219, bem como para leitura tensão, corrente e potência em diferentes escalas. A Fig. 16 apresenta a classe *Adafruit_INA219* contida no repositório do *github* que hospeda o código fonte original do cliente mantido pela *Adafruit*.

Figura 16: Código fonte no arquivo *Adafruit_INA219.cpp*.



Fonte: (ADAFRUIT, 2024).

Para estabelecer a comunicação entre o INA219 e o ESP32 por meio do protocolo I2C, é necessário conhecer o endereço I2C do INA219, o qual é definido no módulo da *Adafruit* pelos jumpers A0 e A1. Estes jumpers, devem ser conectados por solda para configurar o endereço.

Tabela 1: Mapeamento entre endereços e posições dos jumpers no INA219.

PLACA	Endereço I2C	Jumpers
1	0x40	Sem solda
2	0x41	Solda em A0
3	0x44	Solta em A1
4	0x45	Solda em A0 e A1

Fonte: (DIYIOT, 2024).

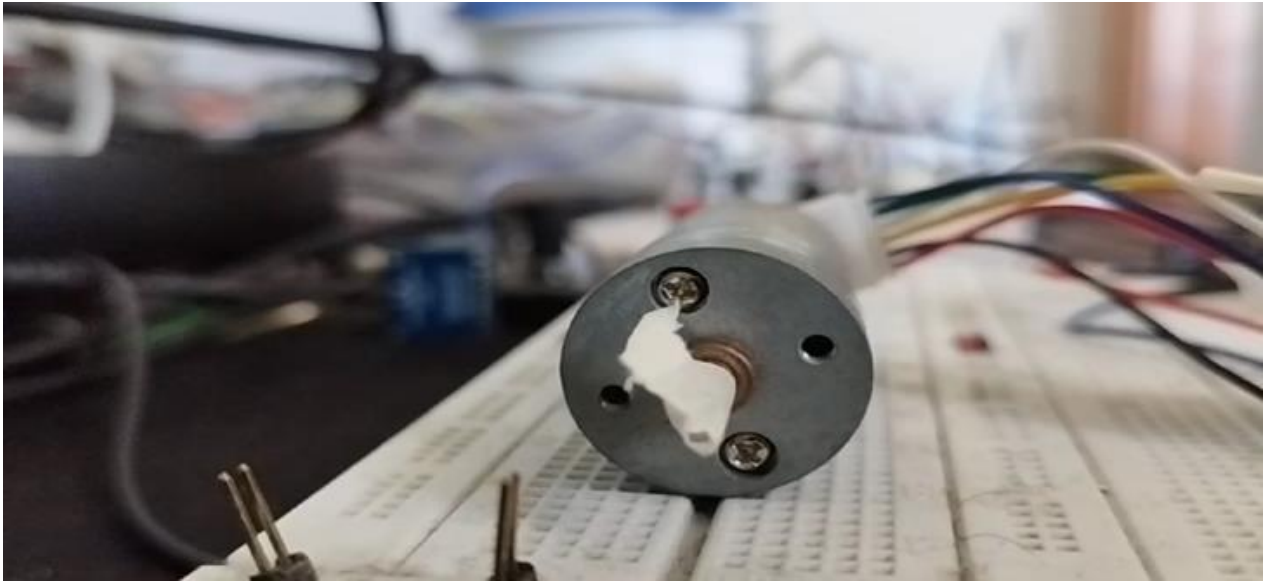
A Tabela 1 apresenta a correspondência entre os endereços I2C e a configuração dos jumpers. Por padrão, utiliza-se o endereço 0x40, que corresponde à situação em que nenhum jumper é soldado. No entanto, em cenários que exigem a conexão de múltiplas placas INA219 ao ESP32, cada dispositivo precisa ser configurado com um endereço único, garantindo que não haja conflito na comunicação (DIYIOT, 2024).

3.8. Definição do número de pulsos por rotação do motor DC

Devido à ausência de informações sobre a especificação do número de pulsos por revolução do motor, foi necessário realizar uma série de procedimentos experimentais para determinar esse parâmetro essencial para a medição da velocidade de rotação. Inicialmente, foi necessário estimar o tempo que o motor leva para completar uma única rotação. Para isso, adotou-se uma abordagem visual. Nesta abordagem, programou-se o *sketch* do Apêndice 2 para gerar um sinal PWM com *duty cycle* em 80% (valor decidido arbitrariamente de modo a diminuir a velocidade de rotação) e utilizando uma câmera de celular configurada no modo *slow motion*. A opção pela utilização do modo *slow motion* foi feita pois esta configuração permite registrar os vídeos com uma taxa de quadros por segundo superior àquela disponível na gravação comum.

Com o uso do modo *slow motion* e o auxílio de uma fita adesiva fixada na haste do motor, foi possível registrar algumas rotações completas do motor. A fita na haste do motor serviu como um marcador visual para facilitar a identificação das rotações no vídeo capturado. A configuração experimental adotada está ilustrada na Fig. 17, que apresenta um frame do vídeo utilizado para a realização dos testes.

Figura 17: Frame extraído do vídeo utilizado para cálculo do tempo de uma única rotação.



Fonte: Autor.

Com o auxílio de uma ferramenta de edição de vídeo, realizaram-se os cortes necessários para remover os quadros excedentes, mantendo apenas os quadros correspondentes a quatro voltas completas, iniciando-se a partir de uma posição arbitrariamente definida da haste. Após a edição, e com a retenção exclusiva dos quadros das quatro voltas completas, extraiu-se a duração total do vídeo, em milissegundos, contendo as quatro revoluções completas.

O tempo total do vídeo com as quatro voltas foi de 11200 milissegundos. A partir da informação do tempo de quatro rotações do motor, calculou-se o tempo médio, em milissegundos, por volta utilizando a seguinte equação:

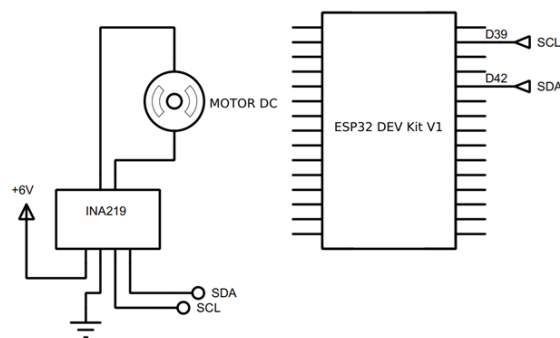
$$Tempo_{rotacao} = \frac{11200}{4} = 2800 \quad (3)$$

Em seguida, para determinar o número de pulsos por revolução, foi desenvolvido o Sketch do Apêndice 3 que foi programado para realizar a leitura dos pulsos gerados pelo motor durante o intervalo de uma rotação completa. A partir dessa medição, obteve-se o valor aproximado de pulsos por revolução. O número médio registrado pelo sketch foi cerca de 812 pulsos por revolução. Para validar esse valor, foi realizada uma consulta às tabelas técnicas de motores de modelo parecido disponíveis no mercado, o que levou à adoção do valor comercial de 823,1 pulsos por revolução.

3.9. Configuração do sensor INA219

Para a configuração do sensor INA219 foi utilizada a sua topologia típica apontada no *datasheet* da fabricante *Texas Instruments*. Nesta topologia, o sensor é adicionado ao circuito do motor de modo a fazer com que seu resistor de *shunt* seja percorrido pela mesma tensão da carga sendo medida. Nesta topologia o sensor será capaz de ler a tensão de barramento (*bus voltage*). A Fig. 18 demonstra o diagrama esquemático simplificado (simplificando as conexões do *encoder* acoplado ao motor) utilizado para os primeiros testes com o sensor INA219.

Figura 18: Topologia de testes para o sensor INA219.



Fonte: Autor.

Nesta montagem, além dos pinos de alimentação, os sinais SCL e SDA são utilizados na comunicação com o microcontrolador. No protocolo I2C, o sinal de SCL é o responsável por fazer a sincronia entre o mestre e os demais dispositivos dentro do mesmo barramento. Ele também determina a velocidade a qual os dados vão trafegar no barramento. Ao passo que o pino SDA é o pino de transmissão dos dados, utilizado tanto para envio quanto recebimento de dados.

Após a montagem do sistema, foram realizados testes utilizando o *cliente* da *Adafruit* para estabelecer a comunicação com o sensor, integrando-o à plataforma Arduino. Durante esses testes, foram feitas medições da tensão, corrente e potência consumidas pelo motor DC. Os resultados das leituras foram enviados em tempo real para o monitor serial do Arduino IDE utilizando o *script* do Apêndice 1. Além disso, os valores de tensão registrados pelo sistema foram comparados com as medições obtidas por um multímetro, o qual foi conectado em paralelo ao motor, a fim de validar a precisão das medições realizadas pela plataforma de aquisição de dados.

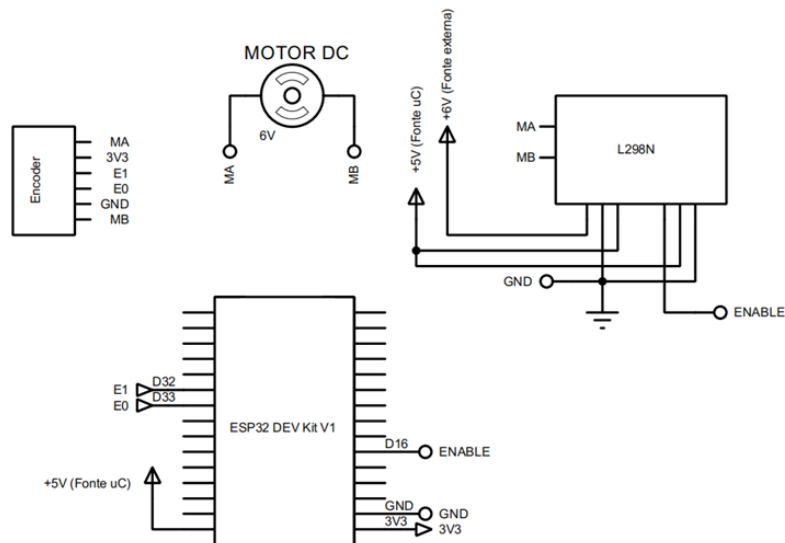
3.10. Configuração da ponte H L298N

Os testes do módulo de ponte H seguiram um procedimento semelhante ao utilizado para o módulo INA219. Inicialmente, foi montada uma topologia simplificada para avaliar o funcionamento

do módulo. Esses testes seguiram a configuração de referência apresentada pela *STMicroelectronics* em seu *datasheet*. Durante esses testes, foi possível validar o funcionamento adequado da ponte H, bem como observar a variação na velocidade de rotação do motor, que ocorria à medida que o sinal PWM enviado para o pino de ENABLE do módulo era ajustado.

Com a mesma configuração experimental, foram realizados testes adicionais utilizando o *encoder* acoplado ao motor para medir a velocidade e a rotação. Para validar o funcionamento correto do sistema de contagem de pulsos, foi necessário programar uma rotina no ESP32, que seria executada toda vez que uma mudança de nível lógico (acionada por borda de subida) fosse detectada em um dos pinos configuráveis como interrupção externa. Para esse fim, foram utilizados os pinos D32 e D33 do microcontrolador como gatilhos para a interrupção, permitindo o incremento de uma variável contadora de pulsos emitidos pelo *encoder* durante as revoluções do motor. O *setup* utilizado para estes testes está descrito na Fig. 19.

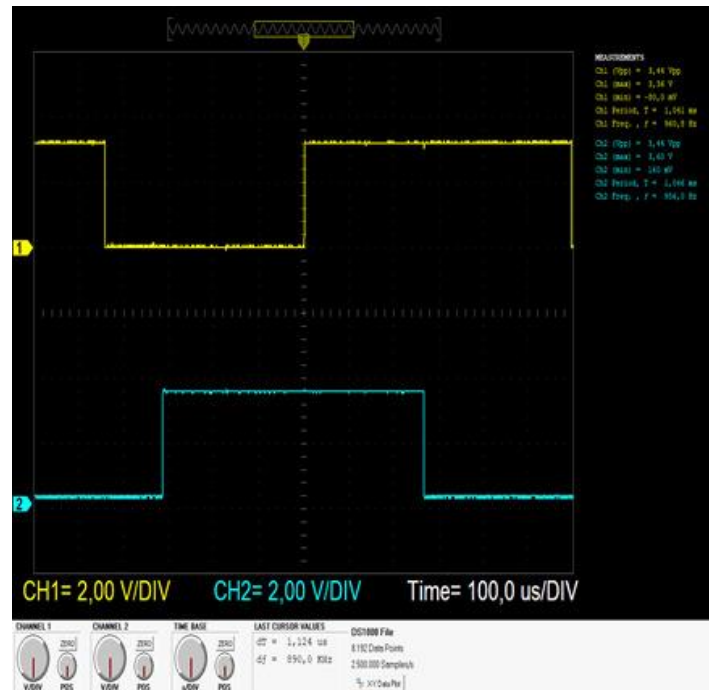
Figura 19: Topologia utilizada para os testes do módulo de ponte H L298N



Fonte: Autor.

Na montagem apresentada na Fig. 19, os pinos MA e MB são responsáveis pela alimentação do motor DC, enquanto os pinos 3V3 e GND são utilizados para alimentar o encoder. Já os pinos E0 e E1 são os pinos de saída, responsáveis pela emissão dos pulsos gerados à medida que o motor realiza suas rotações. Esses pulsos são utilizados para monitorar o movimento do motor e calcular a sua posição ou velocidade. A fig. 20 demonstra o sinal obtido da leitura destes pulsos em um osciloscópio digital.

Figura 20: Ondas do sinal gerado pelo encoder durante a rotação do motor.



Fonte: Autor.

Os sinais de saída do *encoder* são enviados para os pinos de interrupção D32 e D33 do ESP32, os quais são responsáveis por acionar a rotina de interrupção externa. O pino D16, por sua vez, está configurado como um gerador de PWM, enviando o sinal para o pino ENABLE do módulo L298N. No módulo L298N, além de receber o sinal PWM proveniente do pino D16 do ESP32, os pinos IN1 e IN2 estão conectados diretamente à fonte de alimentação, uma vez que não está sendo controlada a direção de rotação do motor (sentido horário ou anti-horário). Os pinos de alimentação do L298N são conectados a uma fonte externa de 6V, enquanto os pinos MA e MB estão ligados à alimentação do motor DC, fornecendo a tensão necessária para o seu funcionamento.

Após a validação prática do comportamento do sensor INA219 e do encoder rotativo de forma individual, foi necessário integrar ambas as montagens em um único sistema.

3.11. Configuração das plataformas InfluxDB e Grafana

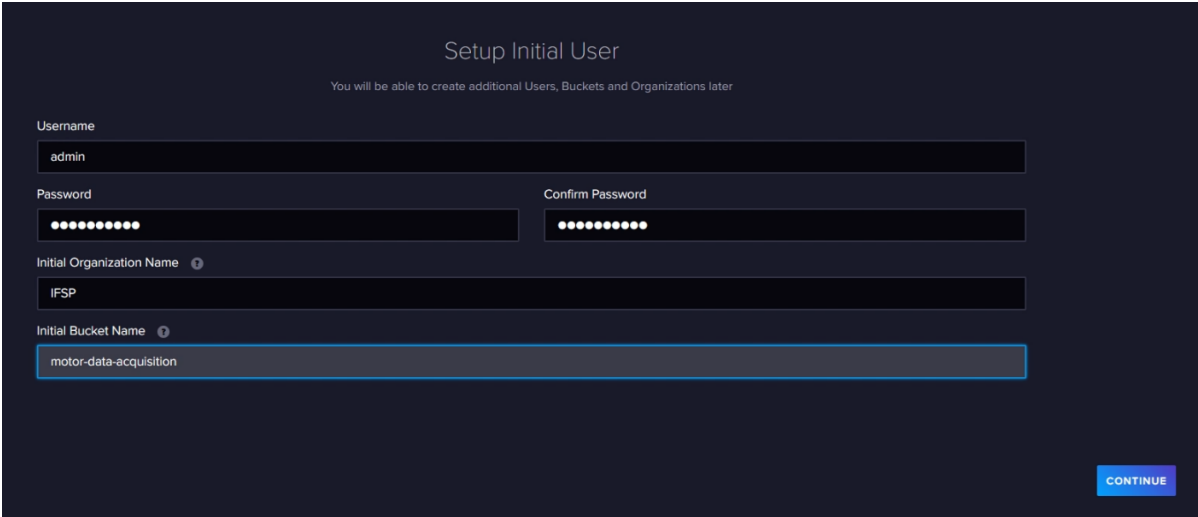
Embora o *InfluxDB* e o *Grafana* sejam ferramentas de código aberto (*open-source*), as principais empresas responsáveis por essas soluções adotam um modelo de negócios baseado na venda de serviços de infraestrutura. Essas empresas oferecem serviços de *cloud* nos quais os usuários podem se conectar, tanto para enviar dados quanto para acessar seus *dashboards*, sem a necessidade de manter servidores próprios. Contudo, esse modelo de serviço está sujeito a custos relacionados ao uso da infraestrutura em nuvem.

Para evitar custos com serviços em nuvem, optou-se por configurar um ambiente local, enviando os dados para uma máquina na rede interna que hospeda tanto o *InfluxDB* quanto o *Grafana*. Dessa forma, toda a coleta e visualização dos dados são realizadas sem a necessidade de utilizar infraestrutura em nuvem.

Entre as diversas opções disponíveis para executar o *InfluxDB* e o *Grafana*, a solução escolhida foi a utilização de contêineres *Docker*. Nesse modelo, cada processo é gerenciado por meio de um arquivo chamado *docker-compose*, que define a configuração e o comportamento dos contêineres. O *docker-compose* especifica quais imagens devem ser utilizadas para construir os contêineres, quais diretórios devem ser mapeados e configura o mapeamento das portas entre a máquina *host* e os contêineres. O *docker-compose* utilizado para subida do *InfluxDB* e do *Grafana* pode ser obtido no Apêndice 6.

Uma vez que o *docker-compose* é executado, ele irá iniciar e expor dois contêineres. O contêiner do *InfluxDB* será exposto na porta padrão 8086, enquanto o contêiner do *Grafana* será exposto na porta 3000. O acesso a ambos os serviços pode ser feito através de um navegador de internet, utilizando o endereço de *loopback* (127.0.0.1), seguido da porta mapeada no *host*.

Figura 21: Tela de setup de usuário inicial no InfluxDB.

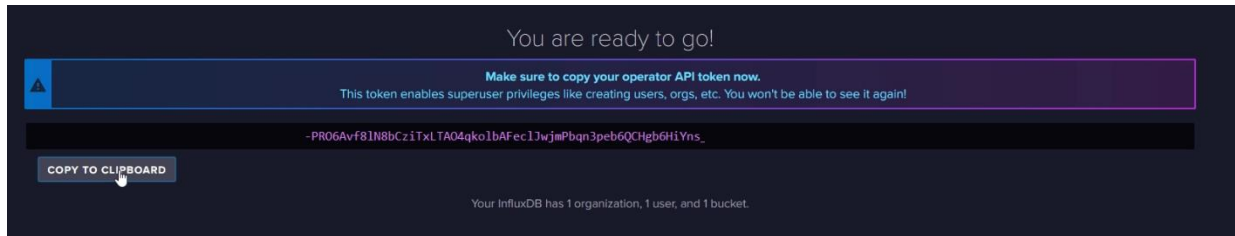


Fonte: Autor.

Como descrito anteriormente, para realizar operações na API do *InfluxDB*, é necessário fornecer um *token* de autorização. Este *token* é gerado no primeiro acesso ao *InfluxDB*. Ao acessar o *InfluxDB* na porta 8086 e efetuar o *login*, o usuário é solicitado a preencher um formulário com informações básicas sobre a organização e o nome do *bucket* inicial. A Fig. 21 ilustra esse processo. Após o preenchimento do formulário, o usuário recebe o *token* de autenticação, que será utilizado nas

chamadas à API do *InfluxDB*. A Fig. 22 apresenta a tela onde o *token* de autorização é exibido, que é a visualização esperada pelo usuário durante esse procedimento.

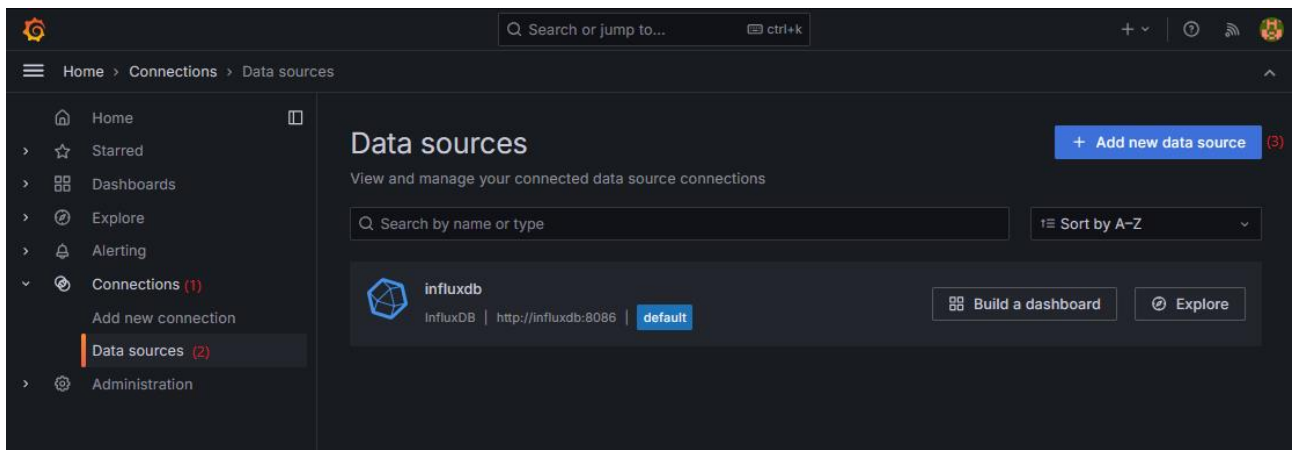
Figura 22: *Token* de autenticação do InfluxDB.



Fonte: Autor.

Uma vez obtido o *token* de autorização é necessário configurar a comunicação entre o Grafana e o InfluxDB. Para isso, deve-se acessar o *Grafana*, que está sendo executado na porta 3000. Após acessar o *Grafana*, é necessário configurar o *InfluxDB* como *datasource*, para que o Grafana consiga realizar as consultas aos dados enviados pelo ESP32. Para configurar um *datasource* no Grafana é preciso acessar, no menu lateral, *connections*, *datasources* e então clicar em *add new datasource*. A Fig. 23 ilustra o caminho para adicionar um novo *datasource*.

Figura 23: Caminho para adição de datasources.



Fonte: Autor.

Neste projeto, como foi utilizado o *docker-compose*, ambos os contêineres (*Grafana* e *InfluxDB*) são criados dentro de uma rede isolada do *Docker*. Dentro dessa rede, o nome do contêiner é usado para resolver o DNS, direcionando para o endereço IP interno do contêiner. Assim, ao configurar a URL base para que o Grafana realize as consultas, é necessário especificar o nome e a porta do contêiner onde o InfluxDB está sendo executado.

Além disso, o InfluxDB oferece suporte a consultas em várias linguagens. No entanto, neste projeto, optou-se pelo uso do InfluxQL, razão pela qual essa opção foi selecionada durante a configuração. Os dados do banco de dados são os mesmos definidos no arquivo *docker-compose* apresentado no Apêndice 6. O usuário a ser utilizado é o mesmo previamente configurado no InfluxDB, e a senha corresponde ao *token* de autorização gerado pelo InfluxDB para autenticação. A Fig. 24 ilustra como foi feita a configuração do InfluxDB como *datasource* dentro do Grafana.

Figura 24: Configuração do InfluxDB como datasource no Grafana.

The image shows the Grafana configuration page for a new data source. The 'Query language' dropdown is set to 'InfluxQL'. Under the 'HTTP' section, the 'URL' is 'http://influxdb:8086', 'Allowed cookies' is 'New tag (enter key to add)' with an 'Add' button, and 'Timeout' is 'Timeout in seconds'. Below this, the 'Database' is 'mydb', 'User' is 'admin', and 'Password' is masked with dots. The 'HTTP Method' dropdown is set to 'Choose', 'Min time interval' is '10s', and 'Max series' is '1000'.

Fonte: Autor.

3.12. Configuração do cliente do InfluxDB no ESP32

O InfluxDB fornece bibliotecas de clientes oficiais para linguagens como *Go*, *Python*, *Java* e outras. Essas bibliotecas frequentemente oferecem recursos avançados, como agrupamento automático, tentativas de reenvio e outras otimizações. O uso das bibliotecas clientes pode ajudar a gerenciar melhor a utilização de recursos, simplificar o código e garantir que as melhores práticas sejam seguidas (HERMANS, 2023). Deste modo optou-se pela utilização do cliente recomendado pela InfluxData para Arduino.

O cliente empregado para a comunicação com o *InfluxDB* demanda uma série de configurações iniciais para garantir seu funcionamento adequado. A primeira etapa consiste na configuração do módulo cliente, para a qual foi consultada a documentação oficial do componente, a fim de identificar os parâmetros necessários para estabelecer a conexão com o banco de dados. No processo de construção do objeto de comunicação, é necessário fornecer informações essenciais, como a URL do servidor onde o *InfluxDB* está em execução, um *token* de autenticação para garantir a segurança da comunicação, o identificador da organização responsável pelo banco de dados, bem

como o nome do *bucket* no qual os dados serão armazenados. A Fig. 22 demonstra a tela de obtenção do *token* de autorização do *InfluxDB*. O Apêndice 1 demonstra um trecho do código de configuração do cliente do *InfluxDB* para a plataforma Arduino.

3.13. Definição da taxa de amostragem

O teorema da amostragem estabelece que um sinal real cujo espectro é limitado em faixa a B Hz ($X(\omega) = 0$ para $|\omega| > 2\pi B$) pode ser reconstruído exatamente (sem qualquer erro) de suas amostras forem tomadas uniformemente a uma taxa de $F_s > 2B$ amostras por segundo. Em outras palavras, a menor frequência de amostragem é $F_s = 2B$ Hz. (LATHI, 2004). Portanto, como o objetivo é monitorar o comportamento de um motor DC, que após o regime transitório não deve apresentar grandes variações nos sinais, foi definida uma taxa de amostragem de 200 ms para todos os sinais elétricos. Essa frequência de amostragem garante uma captura adequada dos dados sem sobrecarga de informações. Para a rotação do motor, optou-se por consolidar a medida de rotação a cada segundo, devido à facilidade em lidar com essa base de tempo.

3.14. Envio dos dados em lote para o InfluxDB

Embora a taxa de amostragem utilizada corresponda a um total de cinco requisições por segundo - valor que o *InfluxDB* consegue processar sem grandes dificuldades - ao lidarmos com volumes elevados de dados, o *InfluxDB* oferece duas abordagens mais eficientes para a ingestão de dados: *streaming* e *batch*. O envio em *streaming* é vantajoso para aplicações que realizam análise de dados em tempo real. No entanto, quando o volume de dados é extremamente alto, essa abordagem pode sobrecarregar o banco de dados. Por outro lado, a ingestão em *batch* permite agrupar vários pontos de dados em uma única operação de escrita, o que pode ser mais eficiente em termos de largura de banda de rede e uso da CPU (HERMANS, 2023).

Além disso, observou-se que, ao enviar os dados de forma individual, o cliente do *InfluxDB* bloqueava a execução do sketch por intervalos consideráveis de tempo, prejudicando a performance do sistema. Dessa forma, optou-se por adotar a abordagem de envio em *batches* com tamanho máximo de 10 itens por vez.

Em termos de código, cada item é representado por uma *struct* em C++ que contém os valores de tensão, corrente, rotação por segundo, potência e *duty cycle*. A estrutura utilizada para representar cada item foi definida da seguinte forma:

```
struct MetricDatum {
    float Voltage;
    float Current;
    float RotationsPerSecond;
    float Power;
    u_int DutyCycle;
};
```

Cada campo da *struct* armazena um valor relevante para o monitoramento do motor DC, permitindo o acompanhamento e análise dos parâmetros elétricos e de desempenho do motor em tempo real.

Para configurar o cliente do InfluxDB de forma a enviar os dados em *batch*, foram ajustadas as *write options*, configurando o buffer interno para até 10 itens, e as *HTTP options* para reutilizar a conexão. A configuração de reutilização de conexão embora não seja obrigatória é recomendada, pois evitamos abertura de novas conexões desnecessariamente, melhorando o tempo de envio dos dados ao InfluxDB. O trecho de código com estas configurações é o seguinte:

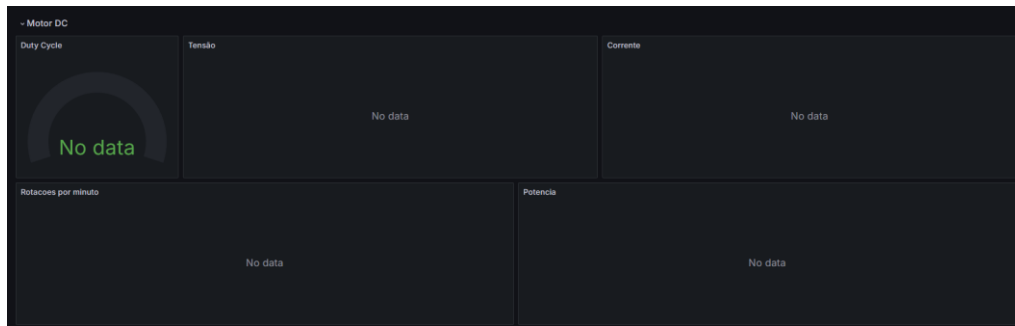
```
void addConfigurationToInfluxDbClient() {
    _client.setWriteOptions(WriteOptions().batchSize(10));
    _client.setHTTPOptions(HTTPOptions().connectionReuse(true));
}
```

3.15. Configuração do *dashboard* no *Grafana*

No *Grafana*, as métricas são exibidas em *dashboards*, que fazem uso dos dados provenientes do *datasource* em suas *views* voltadas ao monitoramento. O formato dessas visualizações varia de acordo com o tipo de métrica a ser apresentada. No contexto deste projeto, as métricas são representadas por indicadores de tipo *gauge*. Para a apresentação das leituras de tensão, corrente, potência e rotações por minuto, foram empregadas visualizações do tipo gráfico, adequadas para monitoramento em tempo real desses parâmetros. Já o valor do duty cycle do motor foi exibido por meio de um *gauge* sem a necessidade de séries temporais. A Fig. 25 demonstra o *dashboard* criado para apresentação destas variáveis.

Uma vez que o *Grafana* permite a exportação dos modelos de seus *dashboards* em um formato JSON, é possível utilizar o conteúdo do Apêndice 7 para importar o *dashboard* e suas visualizações em qualquer outro servidor que esteja executando o *Grafana*.

Figura 25: *Dashboard* montado para apresentar os valores lidos do motor DC.

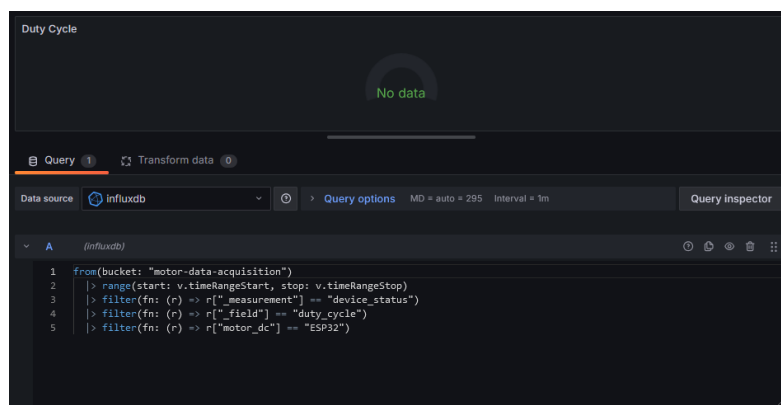


Fonte: Autor.

Para cada *view*, foi necessário escrever uma *query* do *InfluxQL* para realizar a consulta no *InfluxDB*. O resultado da *query* é o que o *Grafana* utiliza para popular as *views*.

A *query* utilizada na exibição do *duty cycle* pode ser vista nos detalhes na Fig. 26.

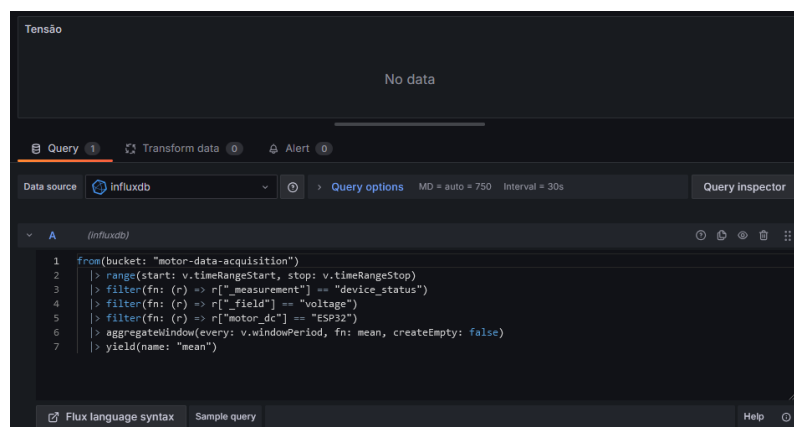
Figura 26: *Query* utilizada para consulta do *duty cycle*.



Fonte: Autor.

A *query* utilizada na exibição da tensão pode ser vista nos detalhes da fig. 27.

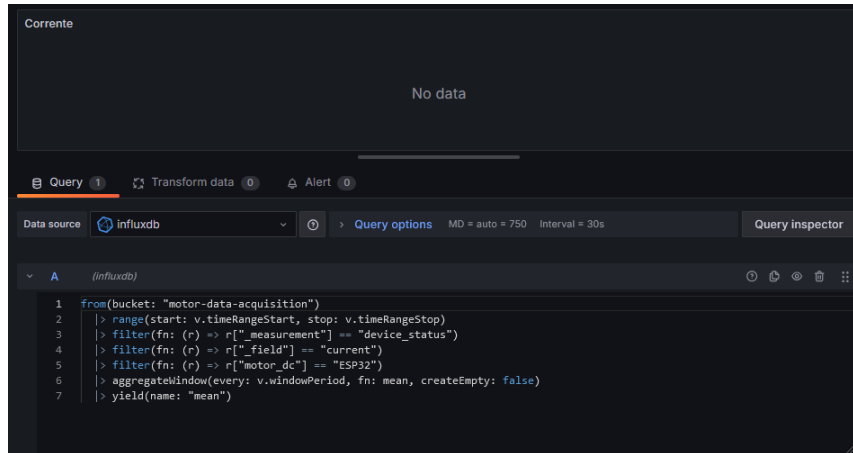
Figura 27: *Query* utilizada para consulta de tensão.



Fonte: Autor.

A query utilizada na exibição da corrente pode ser vista nos detalhes da Fig. 28.

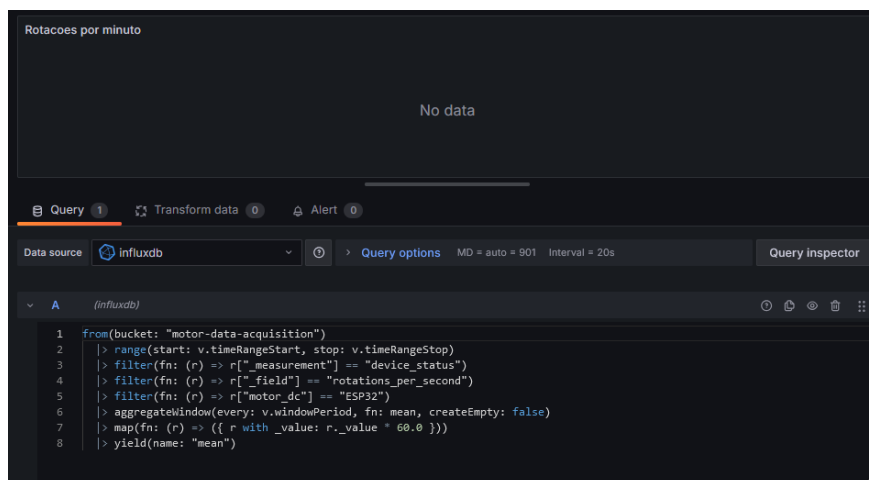
Figura 28: Query utilizada para consulta de corrente elétrica.



Fonte: Autor.

A query utilizada na exibição das rotações por minuto pode ser vista nos detalhes da fig. 29.

Figura 29: Query utilizada para exibição de rotações por minuto.

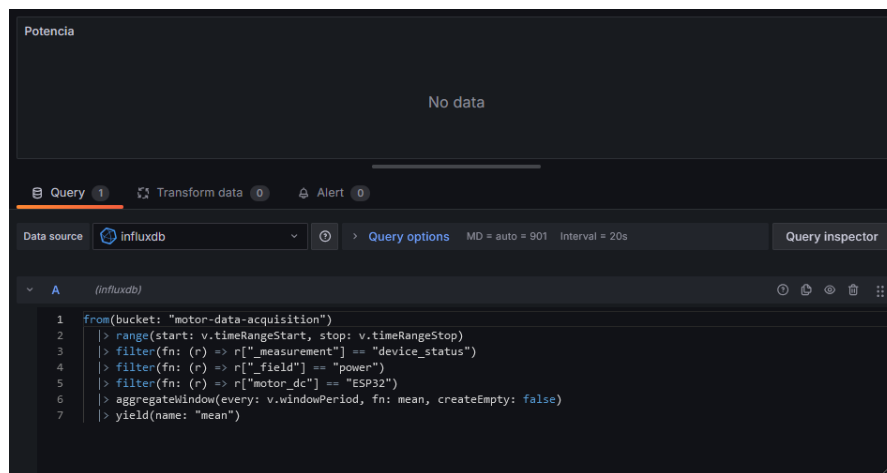


Fonte: Autor.

A query utilizada para exibir as rotações passa por uma pequena transformação para que os dados apresentados na exibição sejam em rotações por minuto (RPM), uma vez que a grandeza é enviada ao InfluxDB no formato de rotações por segundo (RPS). Para realizar essa conversão, multiplica-se o valor das rotações por segundo por 60, a fim de obter o valor correspondente em rotações por minuto.

A query utilizada para a exibição da potência no motor pode ser vista nos detalhes da Fig. 30.

Figura 30: Query utilizada para exibição de potência.

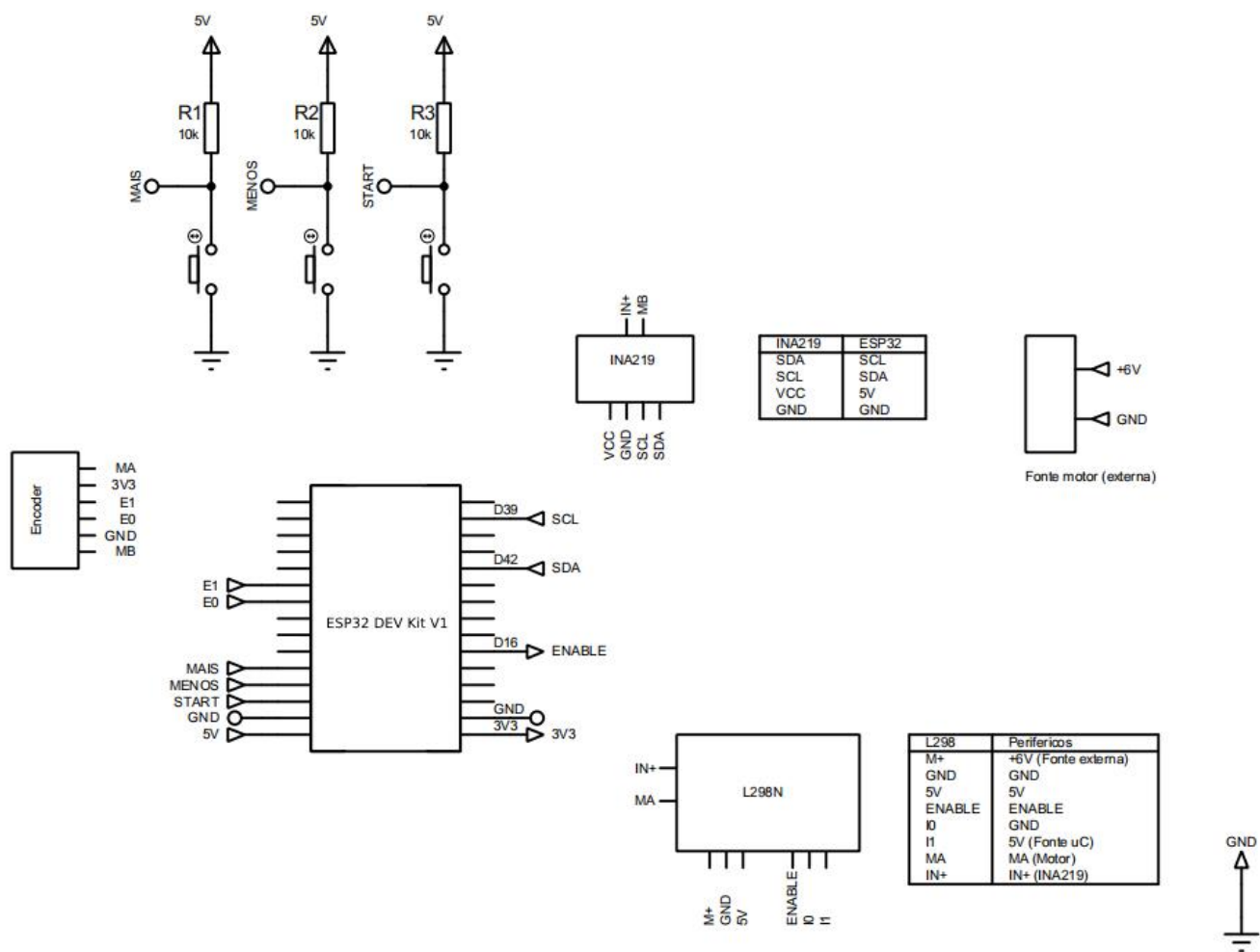


Fonte: Autor.

Após configurar o *InfluxDB*, o *Grafana* e testar o *hardware* interagindo com os sensores e o motor, juntou-se todas as etapas descritas anteriormente para testar a comunicação com o sistema montado. Além do *hardware* descrito anteriormente foram também adicionados três botões (mais, menos e start) para controle do *duty cycle* e da partida do motor. O diagrama da Fig. 31 demonstra o esquemático completo do sistema de aquisição de dados.

Com base no diagrama esquemático da Fig. 31, desenvolveu-se o modelo de placa se circuito impresso da Fig. 32.

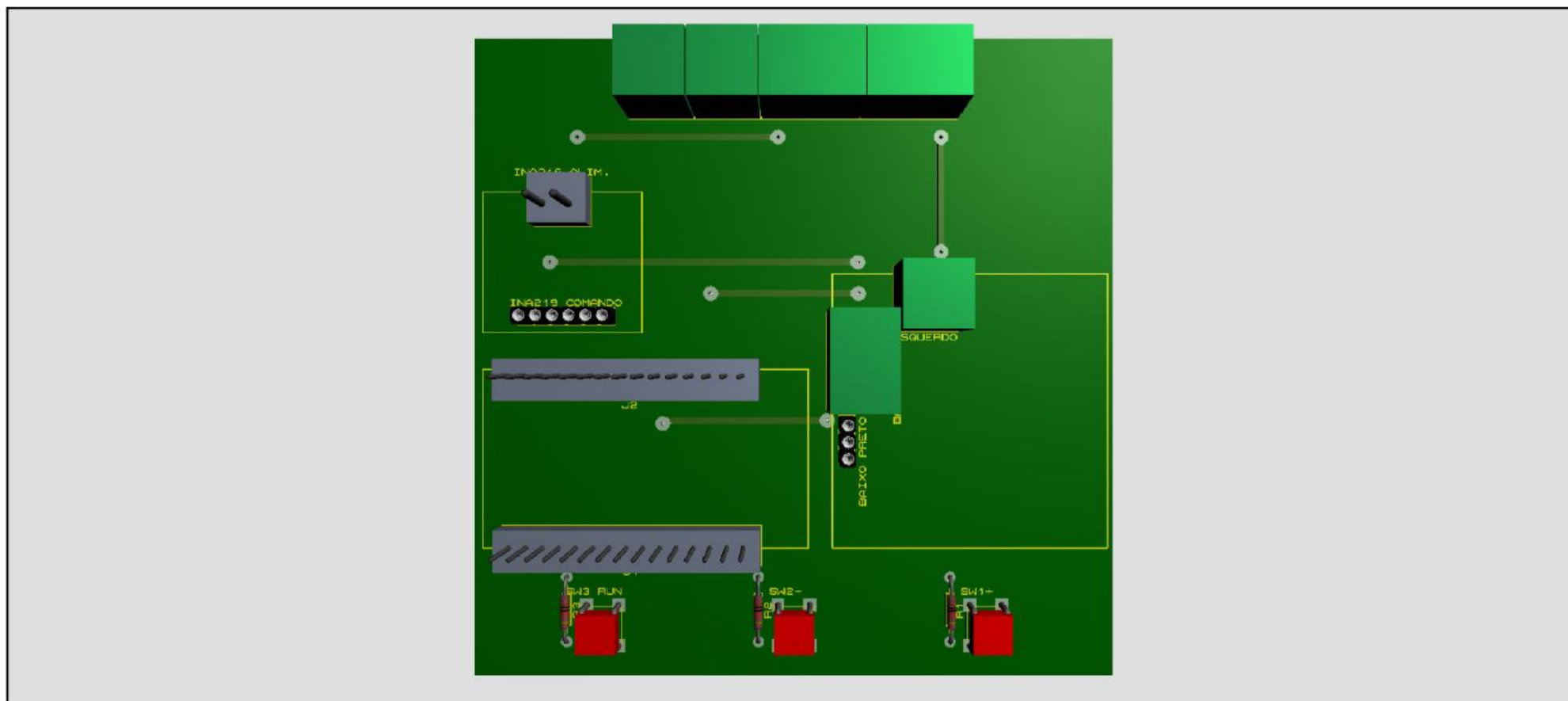
Figura 31: Esquemático completo do sistema de aquisição de dados do motor DC.



OBS: Os pinos Dxx do ESP32 DEV Kit não representam necessariamente a ordem dos pinos do microcontrolador. Considerar a pinagem do ESP32 DEV Kit V1.

Fonte: Autor.

Figura 32: Visualização 3D da placa de circuito projetada para a leitura dos sinais do motor DC.



Fonte: Autor.

O sistema pode ser descrito da seguinte forma: ele utiliza o sensor INA219 para realizar a medição de parâmetros elétricos como tensão, corrente e potência, enquanto o módulo de ponte H L298N é responsável pelo controle e ativação do motor. Quando o motor é acionado e começa a operar, um *encoder*, fixado no corpo do motor, gera pulsos que são capturados pelas entradas E0 e E1 (localizadas nos pinos 32 e 33 do microcontrolador ESP32, respectivamente). Esses pulsos são lidos na borda de subida, acionando uma interrupção no código do microcontrolador, que então incrementa a contagem de pulsos registrados.

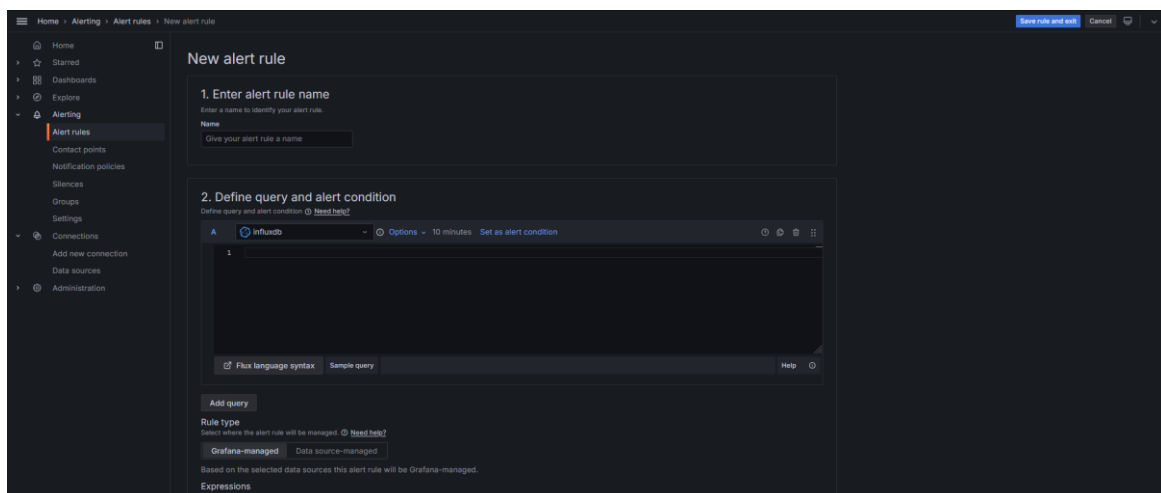
As medições de tensão, corrente e potência são realizadas a cada 200 ms, com os dados sendo temporariamente armazenados em um buffer na memória interna do ESP32. Após um segundo completo de coleta de dados, as informações agregadas são enviadas de forma eficiente ao banco de dados InfluxDB em uma operação em *batch*, visando otimizar o desempenho e a utilização dos recursos do sistema.

Uma vez que os dados são ingeridos no InfluxDB, eles ficam disponíveis para consulta e visualização por meio do *Grafana*, e de sua interface gráfica para monitoramento e análise, permitindo ao usuário acompanhar as variáveis de interesse em tempo real.

3.16. Configuração de alerta no *Grafana*

Para configurar o alerta no *Grafana*, é necessário acessar o painel de alertas a partir do menu lateral, clicando em *Alerting*, depois em *Alert rules* e, por fim, em *New alert rule*. Ao selecionar essa opção, será aberto um painel onde será necessário definir diversos parâmetros para a criação da regra de alerta. A Fig. 33 ilustra o painel de alerta do Grafana.

Figura 33: Painel de configuração de alerta do Grafana.



Fonte: Autor.

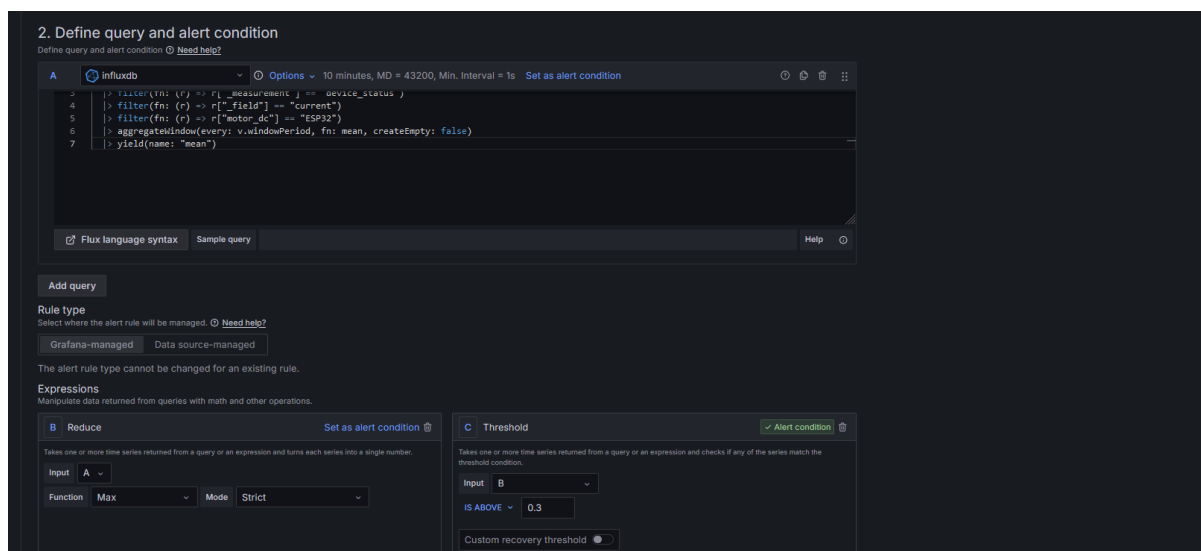
No painel é necessário atribuir um nome à regra de alerta, configurar a *query* que irá consultar os dados a serem monitorados, além de escolher o *datasource* a ser utilizado. Em seguida, é preciso selecionar uma expressão matemática agregadora para processar os dados. Essa expressão será responsável por resumir as informações e calcular o valor da métrica utilizada pelo alerta.

Além da expressão agregadora, é fundamental configurar um limite, que representa o valor máximo esperado da função agregadora em condições normais de operação do sistema. É importante salientar que este valor deve ser bem dimensionado uma vez que o mal dimensionamento pode causar alarmes falsos caso o valor retornado pela função de agregação ultrapasse esse limite.

Adicionalmente, é necessário definir um tempo de avaliação, que é o intervalo entre as avaliações da regra de alerta. Isso determina com que frequência o Grafana irá verificar os dados para verificar se o alerta deve ser acionado. Por fim, é preciso configurar o *contact point*, que especifica quais entidades (como e-mails, tópicos de notificação ou *webhooks*) serão notificadas quando o limite for ultrapassado.

Para este projeto, o alerta configurado foi destinado ao monitoramento da corrente máxima do motor DC, com uma avaliação a cada um minuto. Caso o valor máximo retornado pela consulta ultrapassasse 300 mA, o alerta seria disparado. A Fig. 34 ilustra a tela de configuração do painel de alertas dentro do Grafana, onde é possível visualizar os parâmetros definidos para essa regra. Além disso, como o Grafana oferece suporte à exportação das configurações de alerta no formato YAML (um formato de serialização de dados legível para humanos), o Apêndice 8 contém a definição completa do alerta.

Figura 34: Definição do alerta de corrente.

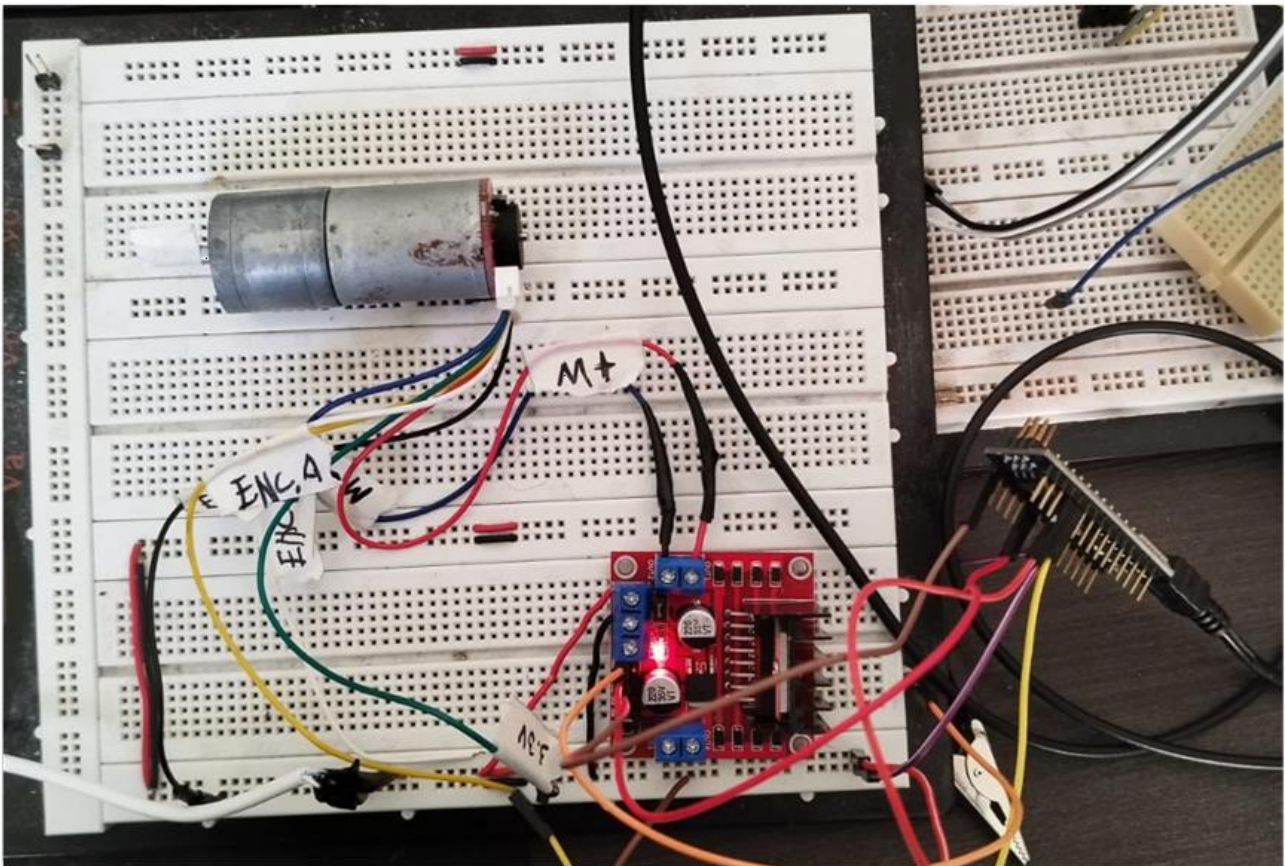


Fonte: Autor.

4. Resultados

Os primeiros testes realizados validaram o acionamento do motor por meio do módulo de ponte H L298N. Para estes testes, foi utilizado um sinal PWM com diferentes valores de *duty cycle*, com o objetivo de verificar o acionamento adequado do motor e testar as conexões com a fonte externa, responsável por fornecer a alimentação ao motor. A Fig. 35 apresenta o setup montado em bancada.

Figura 35: Setup para testes com o módulo L298N.



Fonte: Autor.

Durante os primeiros testes, observou-se que, ao inverter os sinais nos pinos IN1 e IN2 do módulo L298N, o sentido de rotação do motor era alterado. Além disso, constatou-se que a velocidade de rotação do motor variava conforme o valor do *duty cycle* do sinal PWM. Com a diminuição do *duty cycle*, o motor apresentava uma rotação mais lenta. Foi também identificado que, para a frequência de 1kHz, para o motor iniciar seu movimento a partir do estado de repouso, era necessário um *duty cycle* superior ao necessário para manter o motor em operação devido à zona morta.

Para o sensor INA219, foram adquiridas as variáveis de tensão, corrente e potência. Durante esses testes, utilizou-se o cliente fornecido pela *Adafruit* aplicando uma calibração específica que permitiu a medição de tensões de até 32 volts e correntes de até 1 ampere. Todos os dados foram

capturados a uma taxa de 1 leitura por segundo e transmitidos, neste primeiro estágio do experimento, para o monitor serial da IDE do Arduino. A Fig. 36 demonstra os dados de tensão obtidos na saída do monitor serial.

Figura 36: Leituras de tensão, corrente e potência medidas a partir do sensor INA219.

```

11:28:37.213 -> Tensao: 4.52V
11:28:37.213 -> Corrente: 130mA
11:28:37.213 -> Potencia: 587.60mW
11:28:38.194 -> Tensao: 4.51V
11:28:38.194 -> Corrente: 130mA
11:28:38.194 -> Potencia: 586.30mW
11:28:39.206 -> Tensao: 4.51V
11:28:39.206 -> Corrente: 130mA
11:28:39.206 -> Potencia: 586.30mW
11:28:40.217 -> Tensao: 4.51V
11:28:40.217 -> Corrente: 130mA
11:28:40.217 -> Potencia: 586.30mW
11:28:41.227 -> Tensao: 4.50V
11:28:41.227 -> Corrente: 130mA
11:28:41.227 -> Potencia: 585.00mW
11:28:42.208 -> Tensao: 4.52V
11:28:42.208 -> Corrente: 130mA
11:28:42.208 -> Potencia: 587.60mW
11:28:43.217 -> Tensao: 4.53V
11:28:43.217 -> Corrente: 130mA
11:28:43.217 -> Potencia: 588.90mW
11:28:44.232 -> Tensao: 4.51V
11:28:44.232 -> Corrente: 130mA
11:28:44.232 -> Potencia: 586.30mW
11:28:45.226 -> Tensao: 4.53V
11:28:45.226 -> Corrente: 130mA
11:28:45.226 -> Potencia: 588.90mW

```

Fonte: Autor.

Nos testes realizados, verificou-se que as leituras obtidas pelo sensor apresentaram consistência com as medições de tensão e corrente realizadas utilizando um multímetro modelo Minipa Et-1002.

Para a medição da rotação do motor, foi utilizada a interrupção externa do microcontrolador, em conjunto com uma base de tempo de um segundo, escolhida propositalmente para que a leitura da rotação fosse em rotações por segundo, e o monitor serial para exibir a quantidade de pulsos capturados pelas interrupções dentro da base de tempo. O setup utilizado para esses testes foi essencialmente o mesmo configurado para os testes com o módulo L298N e o mesmo setup utilizado para determinar a quantidade de pulsos gerados pelo motor a cada revolução. A Fig. 37 apresenta a interface do monitor serial da IDE do Arduino, exibindo os valores correspondentes às medições dos pulsos.

Figura 37: Saída do *script* de contagem de pulsos.

```

Quantidade de pulsos: 833
Quantidade de pulsos: 822
Quantidade de pulsos: 821
Quantidade de pulsos: 821
Quantidade de pulsos: 814
Quantidade de pulsos: 819
Quantidade de pulsos: 815
Quantidade de pulsos: 813
Quantidade de pulsos: 833
Quantidade de pulsos: 813
Quantidade de pulsos: 818
Quantidade de pulsos: 829
Quantidade de pulsos: 817
Quantidade de pulsos: 817
Quantidade de pulsos: 823
Quantidade de pulsos: 831
Quantidade de pulsos: 832
Quantidade de pulsos: 828

```

Fonte: Autor.

Durante os testes, observou-se que a quantidade de pulsos gerados pelo *encoder* diminuía quando uma força era aplicada ao eixo do motor com a intenção de restringir seu movimento, comportamento que indica uma relação entre a resistência mecânica e a frequência dos pulsos. Além disso, ao inverter os sinais nos pinos IN1 e IN2 do módulo L298N, foi notado que a mudança no sentido de rotação do motor provocava uma alteração no ângulo de fase do sinal gerado pelo *encoder*. Como resultado, o *software* responsável pela captura dos pulsos, que estava inicialmente configurado para o sentido de rotação estabelecido, necessitou de ajustes para compensar essa inversão e refletir corretamente as rotações no sentido contrário.

Após a integração dos sensores INA219, do *encoder* e do módulo de ponte H L298N e ao carregar o *sketch* do Apêndice 10, implementando a comunicação com o banco de dados InfluxDB e iniciando a operação do sistema utilizando o ESP32. Como resultado, ao acessar a interface do Grafana e ativar o sistema, foi possível visualizar, em tempo real, os dados enviados pelo ESP32 diretamente no painel de controle. Além disso, realizou-se a variação da tensão aplicada ao motor, e observou-se que as medições das grandezas, de corrente, tensão, potência e velocidade angular, incluindo, também, a visualização do valor do *duty cycle* do motor. A Fig. 38 exemplifica o comportamento observado durante os testes realizados.

Figura 38: *Dashboard* do motor DC populado com as leituras do motor no Grafana.

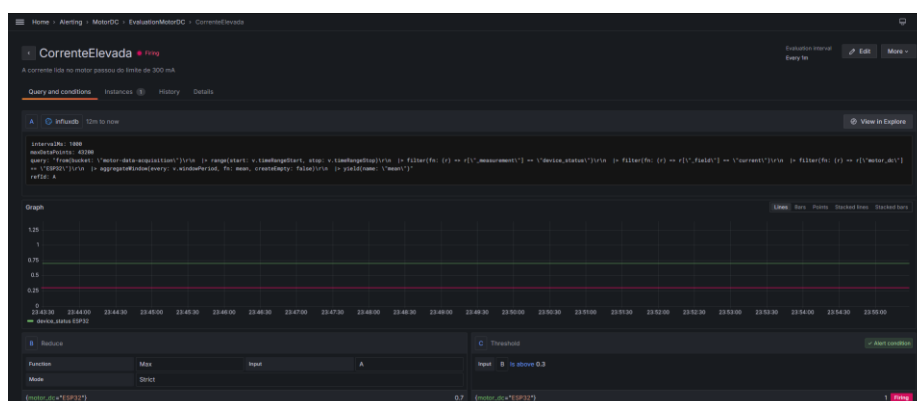


Fonte: Autor.

Os dados exibidos no Grafana foram validados por meio de comparações diretas com as medições realizadas no próprio motor com o auxílio de um multímetro e com o amperímetro acoplado a fonte de alimentação externa do motor. Os resultados obtidos mostraram-se consistentes, apresentando uma correspondência muito próxima entre os dados coletados pelo sistema de aquisição e os valores medidos fisicamente no motor.

O funcionamento do alerta configurado para ser acionado quando a corrente do motor ultrapassasse 300 mA foi testado utilizando-se do *script* C# descrito no Apêndice 9. Este script, criado com o intuito de evitar danos ao motor, simula um sinal de corrente de 700 mA. Ao enviar esses valores para o InfluxDB e após o período de avaliação do alerta, foi possível observar, no Grafana, que o alerta foi devidamente acionado, indicando que a configuração de monitoramento e notificação funcionou como esperado, alertando a ultrapassagem do limite de corrente estabelecido. A Fig. 39 demonstra o alerta acionado no Grafana.

Figura 39: Alerta de corrente elevada disparado no Grafana.



Fonte: Autor.

5. Conclusão

Este trabalho apresentou a implementação de um sistema supervisor com IoT para análise de um Motor DC. O sistema utilizou-se de componentes de baixo custo para fazer a leitura de quatro variáveis a cada 200 ms e enviar os valores lidos para o *InfluxDB*, um banco de dados do tipo TSDB que, posteriormente, é consultado pelo *Grafana* para a montagem de um *dashboard* contendo o histórico das medidas.

Dois sensores foram utilizados para ler a tensão, corrente, potência e quantidade de rotações por segundo do motor, além do módulo para acionamento do motor pelo microcontrolador. Comparativamente, o sistema apresentou valores muito parecidos com os de um equipamento de medição comum, porém, com a vantagem de termos o histórico dos dados lidos, uma interface que apresenta o comportamento através do tempo e a possibilidade de criação de alertas para eventuais falhas na planta.

Devido à natureza do sistema ser microcontrolado, em situações que exigem uma taxa de amostragem extremamente baixa, podem ocorrer perdas de sinal. Isso se deve ao fato de que cada instrução do microcontrolador requer uma ou mais unidades de *ciclo de máquina* para ser executada. Além disso, como os dados são lidos e enviados de forma síncrona, o que, combinado com a natureza bloqueante do protocolo HTTP pode resultar em intervalos de medição inconsistentes, devido ao tempo de espera necessário para o envio de dados. Para mitigar esse problema, foi adotada uma base de tempo suficientemente grande, o que garante uma maior estabilidade entre as amostras. Além disso, foi implementada a técnica de envio das leituras em *batch* para o *InfluxDB*, o que permite agrupar várias leituras e enviá-las de uma só vez, reduzindo a sobrecarga do processo de comunicação e a adoção da reutilização da conexão HTTP entre os envios, o que diminui o impacto bloqueio do protocolo nas medições.

Para eventuais necessidades de tempos de taxa de amostragem menores em sistemas microcontrolados, além das soluções apresentadas neste trabalho, pode-se pensar em otimizar o código executado pelo microcontrolador, diminuindo possíveis processos bloqueantes que ocorram nas bibliotecas dos módulos utilizados. Além disso, a depender do processador utilizado, pode-se considerar separar a leitura e o envio dos dados em núcleos distintos, de forma paralela. Nesse caso, um núcleo ficaria dedicado exclusivamente à leitura dos dados dos sensores, enquanto, após a coleta das informações, os dados seriam colocados em uma fila que suporte concorrência (*concurrent queue*). Um segundo núcleo, então, seria responsável por ler essa fila e realizar o envio das leituras para o *InfluxDB*, gerenciando a natureza bloqueante do protocolo HTTP; impedindo, desta forma, bloqueios na leitura dos sinais.

Este dispositivo, em conjunto com tecnologias como o *InfluxDB* e o *Grafana*, se mostra muito promissor para atender as necessidades tanto da indústria em ter um painel de supervisão e alertas de eventuais falhas quanto para as dificuldades encontradas nas aulas de conversão de energia, onde se faz necessário a leitura e *datalog* dos parâmetros de estudo das máquinas elétricas de corrente contínua.

6. Referências:

ADAFRUIT. Adafruit_INA219.cpp. GitHub, 2024. Repositório de código-fonte. Disponível em: https://github.com/adafruit/Adafruit_INA219/blob/master/Adafruit_INA219.cpp. Acesso em: 18 nov. 2024.

ALEXANDER, Charles K.; SADIKU, Matthew N.O. Fundamentals of Electric Circuits. 5. ed. New York: McGraw-Hill, 2013.

ARDUINO E CIA. Motor DC com encoder embutido. Disponível em: https://www.arduinoecia.com.br/wp-content/uploads/2016/10/Motor_DC_Com_Encoder_Embutido.png. Acesso em: 18 nov. 2024.

DIYIOT. INA219 tutorial for Arduino and ESP. Disponível em: <https://diyi0t.com/ina219-tutorial-for-arduino-and-esp/>. Acesso em: 18 nov. 2024.

DYNAPAR. Optical Rotary Encoders. Disponível em: <https://www.dynapar.com/technology/rotary-optical-encoder/>. Acesso em: 12 nov. 2024.

ELIPSE. Elipse Plant Manager - Eclipse Software. Disponível em: <https://www.elipse.com.br/produto/elipse-plant-manager/>. Acesso em: 18 nov. 2024.

ESP32. Placa de Desenvolvimento ESP32. Disponível em: <https://www.makerhero.com/produto/modulo-wifi-esp32-bluetooth>. Acesso em: 14 Nov. 2024.

ESPRESSIF SYSTEMS. Installing - Arduino ESP32 latest documentation. Documentação, 2024. Disponível em: <https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html>. Acesso em: 12 nov. 2024.

ESPRESSIF SYSTEMS. arduino-esp32. GitHub, 2024. Repositório de código-fonte. Disponível em: <https://github.com/espressif/arduino-esp32>. Acesso em: 18 nov. 2024.

FITZGERALD, A. E.; KINGSLEY, Charles Jr.; UMANS, Stephen D. Máquinas Elétricas. 6. ed. São Paulo: Bookman, 2006.

HALLIDAY, D.; RESNICK, R.; WALKER, J. Fundamentos de física: volume 3: eletromagnetismo. Tradução Ronaldo Sérgio de Biasi. 10. ed. Rio de Janeiro: LTC, 2016.

HASSAN, Q. F.; REHMANI, M. H. Internet of Things: Advances in Cloud Computing Technologies for IoT. 1. ed. Boca Raton: CRC Press, 2020.

HERMANS, Kris. Mastering InfluxDB: A Comprehensive Guide to Learning the InfluxDB Database. Publicado de forma independente, 2023.

HUGHES, Austin E. Electric Motors and Drives: Fundamentals, Types and Applications. 3. ed. Oxford: Newnes, 2006.

INFLUXDATA. InfluxDB 1.x HTTP API. Documentação, 2024. Disponível em: <https://docs.influxdata.com/influxdb/v2/api/>. Acesso em: 12 nov. 2024.

INFLUXDATA. Write data with the InfluxDB API. Disponível em: <https://docs.influxdata.com/influxdb/v2/write-data/developer-tools/api/>. Acesso em: 18 nov. 2024.

LATHI, B. P. Sinais e sistemas lineares. 2. ed. São Paulo: Pearson, 2004.

MOBLEY, R. K. An introduction to predictive maintenance. 1. ed. Boston: Butterworth-Heinemann, 2002.

PALIWAL, Snehal K.; KHARDE, Shaila P. A review on data acquisition and control system for industrial automation application. Disponível em: https://iaeme.com/MasterAdmin/Journal_uploads/IJECET/VOLUME_6_ISSUE_7/40120150607004.pdf. Acesso em: 18 nov. 2025.

ROGGIA, Leandro; CARDOZO, Rodrigo. Automação industrial. Fuentes. Santa Maria: Universidade Federal de Santa Maria, Colégio Técnico Industrial de Santa Maria, Rede e-Tec Brasil, 2016.

SALITURO, Eric. Learn Grafana 10.x - Second Edition: A beginner's guide to practical data analytics, interactive dashboards, and observability. 2. ed. Birmingham: Packt Publishing, 2023.

SCHUERG, Tobias. InfluxDB Client for Arduino. GitHub, 2024. Disponível em: <https://github.com/tobiasschuerg/InfluxDB-Client-for-Arduino>. Acesso em: 12 nov. 2024.

STANKOVIC, J. A. Research directions for the Internet of Things. IEEE Internet of Things Journal, 2014.

STMICROELECTRONICS. L298N: Dual full-bridge driver. Disponível em: <https://www.st.com/resource/en/datasheet/l298.pdf>. Acesso em: 12 nov. 2024.

STALLINGS, W. Data and computer communications. 10. ed. Boston: Pearson, 2014.

TEKTRONIX. Data Acquisition Systems (DAQ). Disponível em: <https://www.tek.com/en/products/keithley/data-acquisition-daq-systems>. Acesso em: 12 nov. 2024.

TEXAS INSTRUMENTS. INA219 Zero-Drift, Bidirectional Current/Power Monitor With I2C Interface. Disponível em: <https://www.ti.com/lit/ds/symlink/ina219.pdf?ts=1731990912540>. Acesso em: 18 nov. 2024.

Apêndice 1 – Sketch de testes do sensor INA219

```
#include <Wire.h>
#include <Adafruit_INA219.h>

Adafruit_INA219 ina219;

void setup() {
  Serial.begin(115200);

  while (!ina219.begin());

  ina219.setCalibration_32V_1A();
}

void loop() {
  float busVoltage = ina219.getBusVoltage_V();
  float current_mA = ina219.getCurrent_mA();
  float power_mW = ina219.getPower_mW();

  Serial.print("Tensao: ");
  Serial.print(busVoltage, 2);
  Serial.println("V");

  Serial.print("Corrente: ");
  Serial.print(current_mA);
  Serial.println("mA");

  Serial.print("Potencia: ");
  Serial.print(power_mW, 2);
  Serial.println("mW");

  delay(1000);
}
```

Apêndice 2 – Sketch utilizado para geração do PWM em 80% para a gravação das rotações

```
#define PWM_PIN 16
#define PWM_FREQUENCY 1000
#define PWM_RESOLUTION 8
#define PWM_DUTY_CYCLE_80_PCT 204

void setup() {
    ledcSetup(0, PWM_FREQUENCY, PWM_RESOLUTION);
    ledcAttachPin(PWM_PIN, 0);

    ledcWrite(0, PWM_DUTY_CYCLE_80_PCT);
}

void loop() {}
```

Apêndice 3 – Sketch utilizado para determinar o número de pulsos por revolução do motor DC

```
#define INTERVAL_MS 2800

int pulseCount = 0;
unsigned long previousMillis = 0;

void onPulse() {
    pulseCount++;
}

void setup() {
    Serial.begin(115200);
    attachInterrupt(digitalPinToInterrupt(33), onPulse, RISING);
}

void loop() {
    unsigned long currentMillis = millis();

    if (currentMillis - previousMillis >= INTERVAL_MS) {
        previousMillis = currentMillis;

        Serial.print("Quantidade de pulsos: ");
        Serial.println(pulseCount);

        pulseCount = 0;
    }
}
```

Apêndice 4 – Sketch utilizado para testes do módulo de ponte H L298N

```
#define INTERVAL_MS 1000
#define ENCODER_INPUT_A 32
#define ENCODER_INPUT_B 33

#define PWM_PIN 16
#define PWM_FREQUENCY 1000
#define PWM_DUTY_CYCLE 255
#define PWM_CHANEL 0

const int resolution = 8;

int encoderChanges = 0;
unsigned long previousMillis = 0;

/*
  The following function was created based on the waveform generated by the
  encoder when the motor was rotating both clock and counter-clockwise.

  The measured waveforms were:

  - When rotating clockwise:
    Encoder B output generates a high-pulse before Encoder A output

    Encoder output A: __|```|__
    Encoder output B: __|```|__

  - When rotating counter-clockwise:
    Encoder A output generates a high-pulse before Encoder B output

    Encoder output A: __|```|__
    Encoder output B: __|```|__
*/
void onEncoderInterruption() {
  int encoderAState = digitalRead(ENCODER_INPUT_A);
  int encoderBState = digitalRead(ENCODER_INPUT_B);

  if (encoderAState == HIGH && encoderBState == LOW) {
    encoderChanges--;
  } else if (encoderAState == LOW && encoderBState == HIGH) {
    encoderChanges++;
  }
}

void setup() {
  Serial.begin(115200);
  attachInterrupt(digitalPinToInterrupt(ENCODER_INPUT_A), onEncoderInterruption,
  RISING);
}
```

```
attachInterrupt(digitalPinToInterrupt(ENCODER_INPUT_B), onEncoderInterruption,
RISING);

ledcSetup(PWM_CHANEL, PWM_FREQUENCY, resolution);
ledcAttachPin(PWM_PIN, PWM_CHANEL);

ledcWrite(0, PWM_DUTY_CYCLE);
}

void loop() {
    unsigned long currentMillis = millis();

    if (currentMillis - previousMillis >= INTERVAL_MS) {
        previousMillis = currentMillis;

        Serial.print("Quantidade de pulsos: ");
        Serial.println(encoderChanges);

        encoderChanges = 0;
    }
}
```


Apêndice 5 – Sketch de exemplo para conexão do cliente do InfluxDB no ESP32

```

#define INFLUXDB_URL "<URL_INFLUX_DB>"
#define INFLUXDB_TOKEN "<TOKEN_ACESSO>"
#define INFLUXDB_ORG "<ID_ORGANIZACAO_INFLUX_DB>"
#define INFLUXDB_BUCKET "motor-data-acquisition"

#define WIFI_SSID "<NOME_REDE_WIFI>"
#define WIFI_PASSWORD "<SENHA_REDE_WIFI>"

#include <InfluxDbClient.h>
#include <InfluxDbCloud.h>

#if defined(ESP32)
    #include <WiFiMulti.h>
    WiFiMulti wifiMulti;
    #define DEVICE "ESP32"
#elif defined(ESP8266)
    #include <ESP8266WiFiMulti.h>
    ESP8266WiFiMulti wifiMulti;
    #define DEVICE "ESP8266"
#endif

// Time zone info
#define TZ_INFO "UTC-3"

InfluxDBClient _client(INFLUXDB_URL, INFLUXDB_ORG, INFLUXDB_BUCKET, INFLUXDB_TOKEN, InfluxDbCloud2CACert);

void ConfigureInfluxDbClient() {
    connectToWifi();
}

void connectToWifi() {
    wifiMulti.addAP(WIFI_SSID, WIFI_PASSWORD);

    while (wifiMulti.run() != WL_CONNECTED) {
        Serial.print(".");
        delay(500);
    }
    Serial.println();

    if (_client.validateConnection()) {
        Serial.print("Connected to InfluxDB: ");
        Serial.println(_client.getServerUrl());
    } else {
        Serial.print("InfluxDB connection failed: ");
        Serial.println(_client.getLastErrorMessage());
    }
}

```

Apêndice 6 – Docker compose utilizado para configuração do Grafana e InfluxDB

```
version: '3.8'

services:
  influxdb:
    image: influxdb:2.7.6-alpine
    container_name: influxdb
    ports:
      - "8086:8086" # InfluxDB default port
    environment:
      - INFLUXDB_DB=mydb
      - INFLUXDB_ADMIN_ENABLED=true
      - INFLUXDB_ADMIN_USER=admin
      - INFLUXDB_ADMIN_PASSWORD=0123456789
    volumes:
      - influxdb_data:/var/lib/influxdb2
    networks:
      - monitoring

  grafana:
    image: grafana/grafana:11.1.4
    container_name: grafana
    ports:
      - "3000:3000" # Grafana default port
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=0123456789
    volumes:
      - grafana_data:/var/lib/grafana
    networks:
      - monitoring
    depends_on:
      - influxdb

volumes:
  influxdb_data:
  grafana_data:

networks:
  monitoring:
```

Apêndice 7 – Modelo JSON utilizado para importação das views no Grafana

```
{
  "annotations": {
    "list": [
      {
        "builtIn": 1,
        "datasource": {
          "type": "grafana",
          "uid": "-- Grafana --"
        },
        "enable": true,
        "hide": true,
        "iconColor": "rgba(0, 211, 255, 1)",
        "name": "Annotations & Alerts",
        "type": "dashboard"
      }
    ]
  },
  "description": "Displays data about motor data acquisition",
  "editable": true,
  "fiscalYearStartMonth": 0,
  "graphTooltip": 0,
  "id": 1,
  "links": [],
  "panels": [
    {
      "collapsed": false,
      "gridPos": {
        "h": 1,
        "w": 24,
        "x": 0,
        "y": 0
      },
      "id": 6,
      "panels": [],
      "title": "Motor DC",
      "type": "row"
    },
    {
      "datasource": {
        "type": "influxdb",
        "uid": "fdvwxhgsuajnk"
      },
      "fieldConfig": {
        "defaults": {
          "color": {
            "mode": "thresholds"
          }
        }
      }
    }
  ]
}
```

```

    "mappings": [],
    "thresholds": {
      "mode": "absolute",
      "steps": [
        {
          "color": "semi-dark-green",
          "value": null
        }
      ]
    },
    "unit": "percent"
  },
  "overrides": []
},
"gridPos": {
  "h": 7,
  "w": 4,
  "x": 0,
  "y": 1
},
"id": 1,
"options": {
  "minVizHeight": 75,
  "minVizWidth": 75,
  "orientation": "auto",
  "reduceOptions": {
    "calcs": [
      "lastNotNull"
    ],
    "fields": "",
    "values": false
  },
  "showThresholdLabels": false,
  "showThresholdMarkers": true,
  "sizing": "auto"
},
"pluginVersion": "11.1.4",
"targets": [
  {
    "datasource": {
      "type": "influxdb",
      "uid": "fdvwxhgsuajnk"
    },
    "query": "from(bucket: \"motor-data-acquisition\")\r\n  |>
range(start: v.timeRangeStart, stop: v.timeRangeStop)\r\n  |> filter(fn: (r) =>
r[\"_measurement\"] == \"device_status\")\r\n  |> filter(fn: (r) => r[\"_fi-
eld\"] == \"duty_cycle\")\r\n  |> filter(fn: (r) => r[\"motor_dc\"] ==
\"ESP32\")",
    "refId": "A"
  }
]

```

```

    }
  ],
  "title": "Duty Cycle",
  "type": "gauge"
},
{
  "datasource": {
    "type": "influxdb",
    "uid": "fdvwkhgsuajnk"
  },
  "fieldConfig": {
    "defaults": {
      "color": {
        "mode": "palette-classic"
      },
      "custom": {
        "axisBorderShow": false,
        "axisCenteredZero": false,
        "axisColorMode": "text",
        "axisLabel": "",
        "axisPlacement": "auto",
        "barAlignment": 0,
        "drawStyle": "line",
        "fillOpacity": 0,
        "gradientMode": "none",
        "hideFrom": {
          "legend": false,
          "tooltip": false,
          "viz": false
        },
        "insertNulls": false,
        "lineInterpolation": "linear",
        "lineWidth": 1,
        "pointSize": 5,
        "scaleDistribution": {
          "type": "linear"
        },
        "showPoints": "auto",
        "spanNulls": false,
        "stacking": {
          "group": "A",
          "mode": "none"
        },
        "thresholdsStyle": {
          "mode": "off"
        }
      },
      "mappings": [],
      "thresholds": {

```

```

        "mode": "absolute",
        "steps": [
            {
                "color": "green",
                "value": null
            },
            {
                "color": "red",
                "value": 80
            }
        ]
    },
    "unit": "volt"
},
"overrides": []
},
"gridPos": {
    "h": 7,
    "w": 10,
    "x": 4,
    "y": 1
},
"id": 2,
"options": {
    "legend": {
        "calcs": [],
        "displayMode": "list",
        "placement": "bottom",
        "showLegend": false
    },
    "tooltip": {
        "mode": "single",
        "sort": "none"
    }
},
"targets": [
    {
        "datasource": {
            "type": "influxdb",
            "uid": "fdvwxhgsuajnk"
        },
        "query": "from(bucket: \"motor-data-acquisition\")\r\n |>
range(start: v.timeRangeStart, stop: v.timeRangeStop)\r\n |> filter(fn: (r) =>
r[\"_measurement\"] == \"device_status\")\r\n |> filter(fn: (r) => r[\"_fi-
eld\"] == \"voltage\")\r\n |> filter(fn: (r) => r[\"motor_dc\"] ==
\"ESP32\")\r\n |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty:
false)\r\n |> yield(name: \"mean\")",
        "refId": "A"
    }
}

```

```

],
"title": "Tensão",
"type": "timeseries"
},
{
  "datasource": {
    "type": "influxdb",
    "uid": "fdvwxhgsuajnk"
  },
  "fieldConfig": {
    "defaults": {
      "color": {
        "mode": "palette-classic"
      },
      "custom": {
        "axisBorderShow": false,
        "axisCenteredZero": false,
        "axisColorMode": "text",
        "axisLabel": "",
        "axisPlacement": "auto",
        "barAlignment": 0,
        "drawStyle": "line",
        "fillOpacity": 0,
        "gradientMode": "none",
        "hideFrom": {
          "legend": false,
          "tooltip": false,
          "viz": false
        },
        "insertNulls": false,
        "lineInterpolation": "linear",
        "lineWidth": 1,
        "pointSize": 5,
        "scaleDistribution": {
          "type": "linear"
        },
        "showPoints": "auto",
        "spanNulls": false,
        "stacking": {
          "group": "A",
          "mode": "none"
        },
        "thresholdsStyle": {
          "mode": "off"
        }
      },
      "mappings": [],
      "thresholds": {
        "mode": "absolute",

```

```

        "steps": [
            {
                "color": "green",
                "value": null
            },
            {
                "color": "red",
                "value": 80
            }
        ]
    },
    "unit": "amp"
},
"overrides": []
},
"gridPos": {
    "h": 7,
    "w": 10,
    "x": 14,
    "y": 1
},
"id": 4,
"options": {
    "legend": {
        "calcs": [],
        "displayMode": "list",
        "placement": "bottom",
        "showLegend": false
    },
    "tooltip": {
        "mode": "single",
        "sort": "none"
    }
},
"targets": [
    {
        "datasource": {
            "type": "influxdb",
            "uid": "fdvwxhgsuajnk"
        },
        "query": "from(bucket: \"motor-data-acquisition\")\r\n |>
range(start: v.timeRangeStart, stop: v.timeRangeStop)\r\n |> filter(fn: (r) =>
r[\"_measurement\"] == \"device_status\")\r\n |> filter(fn: (r) => r[\"_fi-
eld\"] == \"current\")\r\n |> filter(fn: (r) => r[\"motor_dc\"] ==
\"ESP32\")\r\n |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty:
false)\r\n |> yield(name: \"mean\")",
        "refId": "A"
    }
],

```



```

    "title": "Corrente",
    "type": "timeseries"
  },
  {
    "datasource": {
      "type": "influxdb",
      "uid": "fdvwxhgsuajnk"
    },
    "fieldConfig": {
      "defaults": {
        "color": {
          "mode": "palette-classic"
        },
        "custom": {
          "axisBorderShow": false,
          "axisCenteredZero": false,
          "axisColorMode": "text",
          "axisLabel": "",
          "axisPlacement": "auto",
          "barAlignment": 0,
          "drawStyle": "line",
          "fillOpacity": 0,
          "gradientMode": "none",
          "hideFrom": {
            "legend": false,
            "tooltip": false,
            "viz": false
          },
          "insertNulls": false,
          "lineInterpolation": "linear",
          "lineWidth": 1,
          "pointSize": 5,
          "scaleDistribution": {
            "type": "linear"
          },
          "showPoints": "auto",
          "spanNulls": false,
          "stacking": {
            "group": "A",
            "mode": "none"
          },
          "thresholdsStyle": {
            "mode": "off"
          }
        },
        "mappings": [],
        "thresholds": {
          "mode": "absolute",
          "steps": [

```

```

        {
            "color": "green",
            "value": null
        },
        {
            "color": "red",
            "value": 80
        }
    ]
},
"unit": "rotrpm"
},
"overrides": []
},
"gridPos": {
    "h": 7,
    "w": 12,
    "x": 0,
    "y": 8
},
"id": 3,
"options": {
    "legend": {
        "calcs": [],
        "displayMode": "list",
        "placement": "bottom",
        "showLegend": false
    },
    "tooltip": {
        "mode": "single",
        "sort": "none"
    }
},
"targets": [
    {
        "datasource": {
            "type": "influxdb",
            "uid": "fdvwxhgsuajnk"
        },
        "query": "from(bucket: \"motor-data-acquisition\")\r\n |>
range(start: v.timeRangeStart, stop: v.timeRangeStop)\r\n |> filter(fn: (r) =>
r[\"_measurement\"] == \"device_status\")\r\n |> filter(fn: (r) => r[\"_fi-
eld\"] == \"rotations_per_second\")\r\n |> filter(fn: (r) => r[\"motor_dc\"] ==
\"ESP32\")\r\n |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty:
false)\r\n |> map(fn: (r) => ({ r with _value: r._value * 60.0 }))\r\n |> yi-
eld(name: \"mean\")",
        "refId": "A"
    }
],

```

```

    "title": "Rotacoes por minuto",
    "type": "timeseries"
  },
  {
    "datasource": {
      "type": "influxdb",
      "uid": "fdvwxhgsuajnk"
    },
    "fieldConfig": {
      "defaults": {
        "color": {
          "mode": "palette-classic"
        },
        "custom": {
          "axisBorderShow": false,
          "axisCenteredZero": false,
          "axisColorMode": "text",
          "axisLabel": "",
          "axisPlacement": "auto",
          "barAlignment": 0,
          "drawStyle": "line",
          "fillOpacity": 0,
          "gradientMode": "none",
          "hideFrom": {
            "legend": false,
            "tooltip": false,
            "viz": false
          },
          "insertNulls": false,
          "lineInterpolation": "linear",
          "lineWidth": 1,
          "pointSize": 5,
          "scaleDistribution": {
            "type": "linear"
          },
          "showPoints": "auto",
          "spanNulls": false,
          "stacking": {
            "group": "A",
            "mode": "none"
          },
          "thresholdsStyle": {
            "mode": "off"
          }
        },
        "mappings": [],
        "thresholds": {
          "mode": "absolute",
          "steps": [

```

```

        {
            "color": "green",
            "value": null
        },
        {
            "color": "red",
            "value": 80
        }
    ]
},
"unit": "watt"
},
"overrides": []
},
"gridPos": {
    "h": 7,
    "w": 12,
    "x": 12,
    "y": 8
},
"id": 5,
"options": {
    "legend": {
        "calcs": [],
        "displayMode": "list",
        "placement": "bottom",
        "showLegend": false
    },
    "tooltip": {
        "mode": "single",
        "sort": "none"
    }
},
"targets": [
    {
        "datasource": {
            "type": "influxdb",
            "uid": "fdvwxhgsuajnk"
        },
        "query": "from(bucket: \"motor-data-acquisition\")\r\n |>
range(start: v.timeRangeStart, stop: v.timeRangeStop)\r\n |> filter(fn: (r) =>
r[\"_measurement\"] == \"device_status\")\r\n |> filter(fn: (r) => r[\"_fi-
eld\"] == \"power\")\r\n |> filter(fn: (r) => r[\"motor_dc\"] == \"ESP32\")\r\n
|> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)\r\n |>
yield(name: \"mean\")",
        "refId": "A"
    }
],
"title": "Potencia",

```

```
    "type": "timeseries"
  }
],
"refresh": "5s",
"schemaVersion": 39,
"tags": [],
"templating": {
  "list": []
},
"time": {
  "from": "now-6h",
  "to": "now"
},
"timepicker": {},
"timezone": "browser",
"title": "motor-data-acquisition",
"uid": "fdx72pck4m9z4a",
"version": 14,
"weekStart": ""
}
```

Apêndice 8 – Definição do alerta exportado do Grafana no formato YAML

```

apiVersion: 1
groups:
  - orgId: 1
    name: EvaluationMotorDC
    folder: MotorDC
    interval: 1m
    rules:
      - uid: ae4dm45i4nmyod
        title: CorrenteElevada
        condition: C
        data:
          - refId: A
            relativeTimeRange:
              from: 600
              to: 0
            datasourceUid: fdvwkhgsuajnkd
            model:
              intervalMs: 1000
              maxDataPoints: 43200
              query: "from(bucket: \"motor-data-acquisition\")\r\n  |>
range(start: v.timeRangeStart, stop: v.timeRangeStop)\r\n  |> filter(fn: (r) =>
r[\"_measurement\"] == \"device_status\")\r\n  |> filter(fn: (r) => r[\"_fi-
eld\"] == \"current\")\r\n  |> filter(fn: (r) => r[\"motor_dc\"] ==
\"ESP32\")\r\n  |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty:
false)\r\n  |> yield(name: \"mean\")"
            refId: A
          - refId: B
            relativeTimeRange:
              from: 600
              to: 0
            datasourceUid: __expr__
            model:
              conditions:
                - evaluator:
                    params: []
                    type: gt
                  operator:
                    type: and
                  query:
                    params:
                      - B
                  reducer:
                    params: []
                    type: last
                  type: query
            datasource:
              type: __expr__

```

```

        uid: __expr__
        expression: A
        intervalMs: 1000
        maxDataPoints: 43200
        reducer: max
        refId: B
        type: reduce
- refId: C
  relativeTimeRange:
    from: 600
    to: 0
  datasourceUid: __expr__
  model:
    conditions:
      - evaluator:
          params:
            - 0.3
          type: gt
        operator:
          type: and
        query:
          params:
            - C
        reducer:
          params: []
          type: last
        type: query
    datasource:
      type: __expr__
      uid: __expr__
      expression: B
      intervalMs: 1000
      maxDataPoints: 43200
      refId: C
      type: threshold
noDataState: NoData
execErrState: Error
for: 1m
annotations:
  description: ""
  runbook_url: ""
  summary: A corrente lida no motor passou do limite de 300 mA
labels:
  "": ""
isPaused: false
notification_settings:
  receiver: ContatosAlertaMotorDC

```

Apêndice 9 – Script para geração de dados para disparo do alerta em C#

```
using InfluxDB.Client;
using InfluxDB.Client.Writes;

const string orgId = "<ORG_ID>";
const string token = "<TOKEN_AUTORIZACAO>";

var client = InfluxDBClientFactory.Create("http://localhost:8086", token);

var pointData = PointData.Measurement("device_status")
    .Tag("motor_dc", "ESP32")
    .Field("current", 0.7d);

var cancellationTokenSource = new CancellationSource();

Console.CancelKeyPress += (sender, e) =>
{
    cancellationTokenSource.Cancel();
    e.Cancel = true;
};

while (!cancellationTokenSource.Token.IsCancellationRequested)
{
    using var write = client.GetWriteApi();
    write.WritePoint(pointData, "motor-data-acquisition", orgId);
    await Task.Delay(1000, cancellationTokenSource.Token);
}
```


Apêndice 10 – Código fonte final do sistema de aquisição de dados

```
// firmware.ino
#define ACQUIRE_DATA_INTERVAL_MS 200
#define SEND_DATA_INTERVAL_MS 1000

#define PLUS_BUTTON 1
#define MINUS_BUTTON 2
#define START_STOP_BUTTON 3

// Modules
#include "MetricDatum.h"
#include "EncoderInterruption.h"
#include "Pwm.h"
#include "Ina219.h"
#include "Influxdb.h"

MetricDatum metrics;

unsigned long millisAtLastEvent = millis();
unsigned long millisAtDataAcquisition = millis();

// Flags
bool flagPlus = false;
bool flagMinus = false;
bool flagStartStop = false;

bool start = false;

u_int dutyCycle = 0;

MetricDatum* onMetricsRequested();
void onTerminalChanged();
void setCurrentMode();

void tryUpdateDutyCycle();
void tryUpdateStartStop();
void executeFreeRunMode();

void setup() {

    Serial.begin(115200);
    Serial.println("Iniciando projeto");

    Serial.println("Configurando ina219...");
    SetupIna219();

    Serial.println("Configurando InfluxDB...");
    ConfigureInfluxDbClient();
}
```

```

Serial.println("Configurando interrupcoes...");
SetupEncoderInterruption();

Serial.println("Configurando PWM...");
SetupPwm(1000);
SetPwmDuty(dutyCycle);

pinMode(BUILTIN_LED, OUTPUT);
digitalWrite(BUILTIN_LED, HIGH);
}

MetricDatum* onMetricsRequested() {
    metrics.Current = GetCurrentInAmpere();
    metrics.Power = GetPowerInWatts();
    metrics.RotationsPerSecond = GetRotations();
    metrics.Voltage = GetLoadVoltageInVolts();
    metrics.DutyCycle = GetPwmDuty();

    return &metrics;
}

void loop() {
    tryUpdateDutyCycle();
    tryUpdateStartStop();

    if (!start) {
        return;
    }

    executeFreeRunMode();
}

void tryUpdateDutyCycle() {
    if (!digitalRead(PLUS_BUTTON)) {
        flagPlus = true;
    }

    if (digitalRead(PLUS_BUTTON) && flagPlus) {
        flagPlus = false;
        dutyCycle += 10;
    }

    if (!digitalRead(MINUS_BUTTON)) {
        flagMinus = true;
    }

    if (digitalRead(MINUS_BUTTON) && flagMinus) {
        flagMinus = false;
    }
}

```

```

    dutyCycle -= 10;
}

if (dutyCycle > 100) {
    dutyCycle = 100;
}

if (dutyCycle < 0) {
    dutyCycle = 0;
}
}

void tryUpdateStartStop() {
    if (!digitalRead(START_STOP_BUTTON)) {
        flagStartStop = true;
    }

    if (digitalRead(START_STOP_BUTTON) && flagStartStop) {
        flagStartStop = false;
        start = !start;
    }
}

void executeFreeRunMode() {
    if (acquireDataIntervalHasPassed()) {
        AddMeasure(onMetricsRequested());

        millisAtDataAcquisition = millis();
    }

    if (configuredIntervalHasPassed()) {
        SendMetrics();

        millisAtLastEvent = millis();
    }
}

inline bool acquireDataIntervalHasPassed() {
    return (millis() - millisAtDataAcquisition) >= ACQUIRE_DATA_INTERVAL_MS;
}

inline bool configuredIntervalHasPassed() {
    return (millis() - millisAtLastEvent) >= SEND_DATA_INTERVAL_MS;
}

// EncoderInterruption.h
#ifndef __ENCODER_INTERRUPTION_H__
#define __ENCODER_INTERRUPTION_H__

```

```

#if (ARDUINO >= 100)
    #include "Arduino.h"
#else
    #include "WProgram.h"
#endif

void SetupEncoderInter interruption();
float GetRotations();

#endif

// EncoderInter interruption.cpp
#define ENCODER_INPUT_A 32
#define ENCODER_INPUT_B 33

#include "EncoderInter interruption.h"

int encoderChanges = 0;

// Should be checked on the motor specification
const float PULSES_PER_REVOLUTION = 823.1;

/*
    The following function was created based on the waveform generated by the
    encoder when the motor was rotating both clock and counter-clockwise.

    The measured waveforms were:

    - When rotating clockwise:
        Encoder B output generates a high-pulse before Encoder A output

        Encoder output A: __|```|__
        Encoder output B: __|```|__

    - When rotating counter-clockwise:
        Encoder A output generates a high-pulse before Encoder B output

        Encoder output A: __|```|__
        Encoder output B: __|```|__
*/
void onEncoderInter interruption() {

    int encoderAState = digitalRead(ENCODER_INPUT_A);
    int encoderBState = digitalRead(ENCODER_INPUT_B);

    if (encoderAState == HIGH && encoderBState == LOW) {
        encoderChanges--;
    } else if (encoderAState == LOW && encoderBState == HIGH) {
        encoderChanges++;
    }
}

```

```

    }
}

void SetupEncoderInterruption() {

    attachInterrupt(digitalPinToInterrupt(ENCODER_INPUT_A), onEncoderInterruption,
RISING);
    attachInterrupt(digitalPinToInterrupt(ENCODER_INPUT_B), onEncoderInterruption,
RISING);
}

float GetRotations() {

    float changesSinceLastCall = encoderChanges;
    encoderChanges = 0;

    float revolutions = changesSinceLastCall / PULSES_PER_REVOLUTION;

    return revolutions;
}

// Ina219.h
#ifndef __INA_219_H__
#define __INA_219_H__

#if (ARDUINO >= 100)
    #include "Arduino.h"
#else
    #include "WProgram.h"
#endif

void SetupIna219();
float GetCurrentInAmpere();
float GetLoadVoltageInVolts();
float GetPowerInWatts();

#endif

// Ina219.cpp
#include "Ina219.h"

// Adafruit ina219 dependencies
#include <Adafruit_INA219.h>

Adafruit_INA219 ina219;

void SetupIna219() {
    ina219.setCalibration_32V_1A();
    while(!ina219.begin());
}

```

```

}

float GetCurrentInAmpere() {
    return ina219.getCurrent_mA() / 1000.0;
}

float GetLoadVoltageInVolts() {
    return ina219.getBusVoltage_V();
}

float GetPowerInWatts() {
    return ina219.getPower_mW() / 1000.0;
}

// MetricDatum.h
#ifndef __METRIC_DATUM_H__
#define __METRIC_DATUM_H__

#if (ARDUINO >= 100)
    #include "Arduino.h"
#else
    #include "WProgram.h"
#endif

struct MetricDatum {
    float Voltage;
    float Current;
    float RotationsPerSecond;
    float Power;
    u_int DutyCycle;
};

#endif

// Pwm.h
#ifndef __PWM_H__
#define __PWM_H__

#if (ARDUINO >= 100)
    #include "Arduino.h"
#else
    #include "WProgram.h"
#endif

void SetupPwm(u_int frequency);
void SetPwmDuty(u_int value);
u_int GetPwmDuty();

#endif

```

```

// Pwm.cpp
#define PWM_PIN 16
#define PWM_CHANEL 0

#include "Pwm.h"

const int resolution = 8;

u_int currentDutyCycle = 0;

void SetupPwm(u_int frequency) {
    ledcSetup(PWM_CHANEL, frequency, resolution);
    ledcAttachPin(PWM_PIN, PWM_CHANEL);
}

void SetPwmDuty(u_int dutyCycle) {

    u_int value = (int)(255.0 * dutyCycle / 100.0);
    currentDutyCycle = dutyCycle;

    ledcWrite(PWM_CHANEL, value);
}

u_int GetPwmDuty() {
    return currentDutyCycle;
}

// Credentials.h (valores omitidos por segurança)
#ifndef __CREDENTIALS_H__
#define __CREDENTIALS_H__

#define INFLUXDB_URL "<URL_INFLUXDB>"
#define INFLUXDB_TOKEN "<TOKEN_AUTORIZACAO_INFLUXDB>"
#define INFLUXDB_ORG "<ID_ORGANIZACAO_INFLUXDB>"
#define INFLUXDB_BUCKET "motor-data-acquisition"

#define WIFI_SSID "<NOME_REDE_WIFI>"
#define WIFI_PASSWORD "<SENHA_REDE_WIFI>"
#endif

// Influxdb.h
#ifndef __INFLUXDB_H__
#define __INFLUXDB_H__

#include "MetricDatum.h"

void ConfigureInfluxDbClient();
void AddMeasure(MetricDatum *metrics);

```

```

void SendMetrics();
void SendCurrentMode(char currentMode);
#endif

// Influxdb.cpp
#include "Influxdb.h"
#include "Credentials.h"

#include <InfluxDbClient.h>
#include <InfluxDbCloud.h>

#ifdef ESP32
    #include <WiFiMulti.h>
    WiFiMulti wifiMulti;
    #define DEVICE "ESP32"
#elif defined(ESP8266)
    #include <ESP8266WiFiMulti.h>
    ESP8266WiFiMulti wifiMulti;
    #define DEVICE "ESP8266"
#endif

// Time zone info
#define TZ_INFO "UTC-3"

// Internal variables
// Influxdb client iniciado com os valores de "Credentials.h"
InfluxDBClient _client(INFLUXDB_URL, INFLUXDB_ORG, INFLUXDB_BUCKET, INFLUXDB_TOKEN, InfluxDbCloud2CACert);
// Point _sensor("device_status");

// Internal functions
void connectToWifi();
void addConfigurationToInfluxDbClient();

void ConfigureInfluxDbClient() {
    connectToWifi();
    addConfigurationToInfluxDbClient();
}

void AddMeasure(MetricDatum *metrics) {
    Point sensor("device_status");

    sensor.addTag("motor_dc", DEVICE);
    sensor.addField("current", metrics->Current);
    sensor.addField("duty_cycle", metrics->DutyCycle);
    sensor.addField("power", metrics->Power);
    sensor.addField("rotations_per_second", metrics->RotationsPerSecond);
    sensor.addField("voltage", metrics->Voltage);
}

```



```

    _client.writePoint(sensor);
}

void SendMetrics() {
    _client.flushBuffer();
}

void connectToWifi() {
    wifiMulti.addAP(WIFI_SSID, WIFI_PASSWORD);

    while (wifiMulti.run() != WL_CONNECTED) {
        Serial.print(".");
        delay(500);
    }

    Serial.println();

    if (_client.validateConnection()) {
        Serial.print("Connected to InfluxDB: ");
        Serial.println(_client.getServerUrl());
    } else {
        Serial.print("InfluxDB connection failed: ");
        Serial.println(_client.getLastErrorMessage());
    }
}

void addConfigurationToInfluxDbClient() {
    _client.setWriteOptions(WriteOptions().batchSize(10));
    _client.setHTTPOptions(HTTPOptions().connectionReuse(true));

    timeSync(TZ_INFO, "pool.ntp.org", "time.nis.gov");
}

```